Zen Function Spec

# Zen — Function Spec

This document describes the design of **Zen**, the Roku QA / test-automation web console: its architecture, components, HTTP API, data flows, data models, and external integrations.

> **Scope.** This design covers the web application: the **frontend** (React SPA), the **backend** (FastAPI), the **4_runcase** runner it drives, and external integrations. The legacy command-line module `1_create_test_plan` is intentionally out of scope; `5_activate` is covered through the Activate tab (§5.20, §8.8).

## Contents


Contents
1. System Overview
  1.1 Architecture Diagram
  1.2 Request Flow
2. Technology Stack
3. Frontend Design
  3.1 Application shell
  3.2 Tabs and pages
  3.3 Tab-visibility configuration
  3.4 Reusable components
  3.5 API client
4. Backend Design
  4.1 Application & configuration
  4.2 Router wiring & path constants
  4.3 Host / port & frontend configuration
5. HTTP API Reference
  5.1 Device — /v1/device
  5.2 Profile — /v1/profile
  5.3 Run Case — /v1/runcase
  5.4 Roku Test — /v1/rokutest
  5.5 Branch — /v1/branch/automated_tests
  5.6 User — /v1/user
  5.7 TestRail — /v1/testrail
  5.8 Skill — /v1/skill
  5.9 Agent — /v1/agent
  5.10 Session — /v1/session
  5.11 Reference — /v1/reference
  5.12 Config — /v1/config
  5.13 Analyze History — /v1/analyze_history
  5.14 Log Analysis — POST /v1/agent/analyze
  5.15 Cmd / Run Script — /v1/cmd
  5.16 File Transfer — /v1/file_transfer
  5.17 Admin & Account Settings

5.18 Dashboard — /v1/dashboard
5.19 Donut — /v1/donut
5.20 Activate — /v1/activate
5.21 Activate Account — /v1/activate_account
6. AI Assistant Design (RAG)
6.1 Components
6.2 Chat data flow (use_internal=true)
6.3 Search data flow (MCP)
7. LLM Interface & Configuration
7.1 Endpoints
7.2 Authentication
7.3 Chat completion request
7.4 Temperature handling for reasoning models
7.5 Chat completion response
7.6 MCP semantic-search call
7.7 Error handling & retries
7.8 Runtime model switching
7.9 Configuration reference
7.10 Chat call sequence
8. Sequence Diagrams
8.1 Prompt Chat (RAG)
8.2 Run Case
8.3 TestRail Search & Profile Generation
8.4 File Transfer
8.5 Admin Login & Settings
8.6 Dashboard Snapshot
8.7 Donuts: Trigger & Track
8.8 Activate: Connected / Unconnected Mode
9. Test Runner & Data Model (4_runcase)
9.1 Component call relationships
9.2 Profile structure
9.3 Execution modes
9.4 Device config model
10. TestRail Caching
11. External Integrations
12. Cross-Cutting Concerns

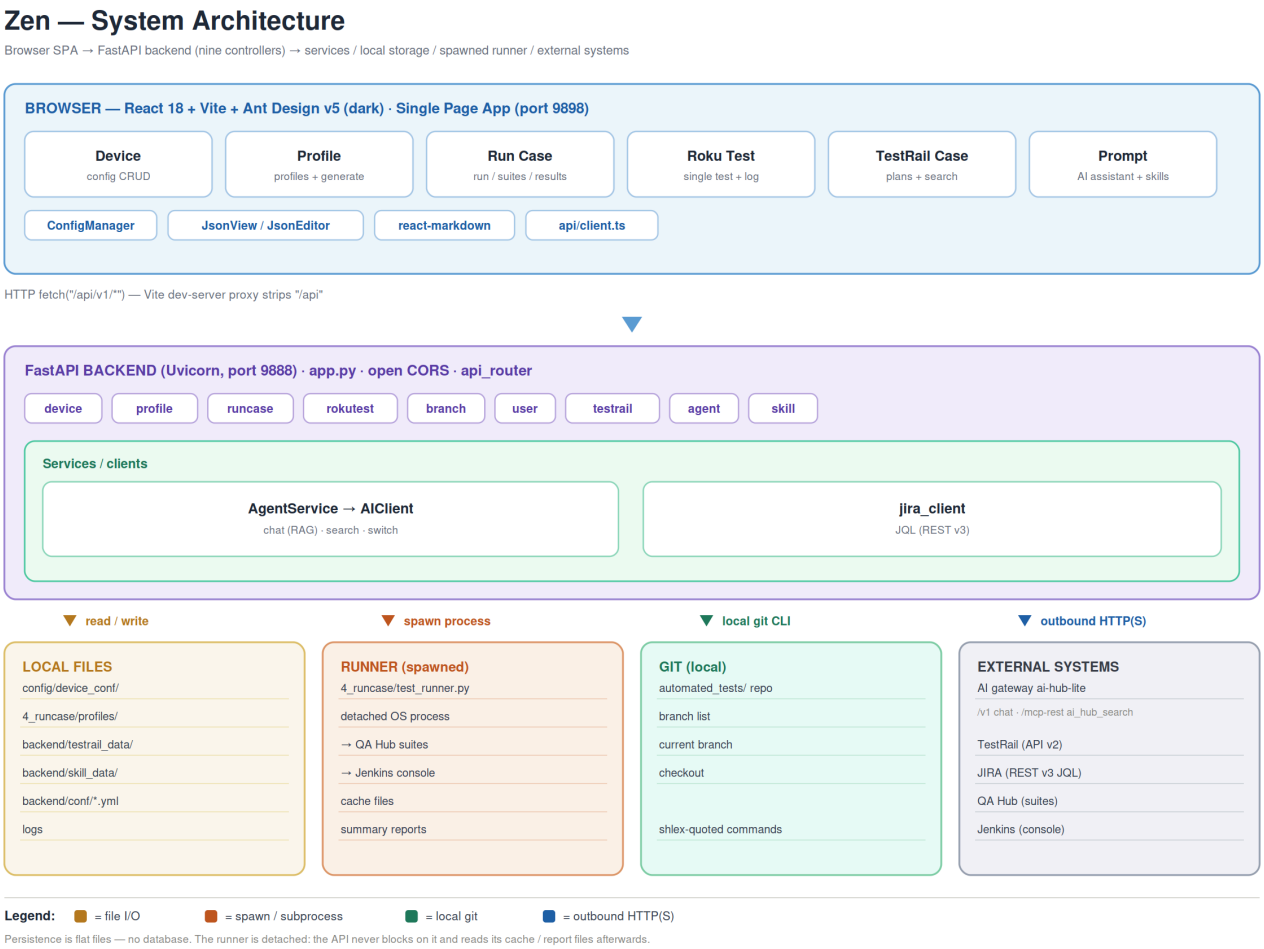
## 1. System Overview

Zen is a two-tier web app plus a set of external services:

- **Frontend** — React 18 + Vite + Ant Design v5 (dark theme), a single-page app served by the Vite dev server on port **9898**. It proxies `/api/*` to the backend.
- **Backend** — FastAPI (Uvicorn) on port **9888**, exposing `/v1/*` REST endpoints. It reads/writes local config files, drives the `4_runcase` test runner, and calls external services.
- **Runner** — `4_runcase/test_runner.py`, launched as a detached OS process per run; it submits suites to QA Hub and writes cache/report files.
- **External services** — AI gateway (ai-hub-lite), TestRail, JIRA, QA Hub, Jenkins, and local git.

### 1.1 Architecture Diagram

End-to-end layered view — browser SPA, the FastAPI backend and its nineteen controllers, the services/clients layer, local storage, spawned runner processes, and external systems.




### 1.2 Request Flow

How a browser action reaches storage and external systems.

**Zen — Request Flow**

```
Browser (SPA :9898)
   |  fetch /api/v1/*        (Vite proxy strips /api)
   v
FastAPI backend (:9888) ── local files: device_conf/, profiles/,
testrail_data/, skill_data/, logs
   |                     ── git CLI on automated_tests/
   |                     ── subprocess: 4_runcase/test_runner.py ──>
QA Hub ──> Jenkins
   |                     ── HTTP:
   |                           AI gateway  (ai-hub-lite /v1  + /mcp-
rest ai_hub_search)
   |                           TestRail    (API v2)
   |                           JIRA        (REST v3 JQL)
   |                           QA Hub      (suites)
```

## 2. Technology Stack

| Layer | Technology |
|---|---|
| Frontend | React 18, TypeScript, Vite 8, Ant Design v5 ( `darkAlgorithm` ), react-markdown + remark-gfm |
| Backend | Python 3, FastAPI, Uvicorn, httpx (async), PyYAML, urllib |
| AI | OpenAI-compatible gateway (ai-hub-lite), MCP tool `ai_hub_search` |
| Persistence | Flat files: JSON (devices, profiles, TestRail cache), Markdown (skills), YAML (config, testrail plans, users) |

## 3. Frontend Design

### 3.1 Application shell

`App.tsx` renders an Ant Design `Layout` : a **Header** ("Zen @kz" brand), a left **Sider** menu, and a **Content** card. It is a single-page app with no router — clicking a tab swaps a `page` state value. The theme is dark ( `theme.darkAlgorithm` via `ConfigProvider` ).

### 3.2 Tabs and pages

| Key | Label | Component | Backend base |
|---|---|---|---|
| `dashboard` | Dashboard | DashboardPage (default landing page) | `/v1/dashboard` |
| `device` | Device | DevicePage → ConfigManager | `/v1/device` |
| `profile` | Profile | ProfilePage → ConfigManager + generator | `/v1/profile` |
| `runcase` | Run Case | RunCasePage | `/v1/runcase` |

| | | | |
|---|---|---|---|
| `rokutest` | Roku Test | RokuTestPage | `/v1/rokutest`, `/v1/branch` |
| `testrail` | TestRail Case | TestRailCasePage | `/v1/testrail` |
| `agent` | Prompt | AgentPage | `/v1/agent`, `/v1/skill` |
| `donuts` | Donuts | DonutsPage | `/v1/donut` |
| `activate` | Activate | ActivatePage | `/v1/activate` |

#### 3.3 Tab-visibility configuration

Tab visibility is data-driven. `frontend/conf/config.yml` has a `tabs:` map (`dashboard/device/profile/runcase/rokutest/analyzehistory/testrail/agent/reference/filetransfer/donuts/activate` → boolean; the runtime order/visibility is managed in `backend/conf/tabs_data.yml` via Settings → Tab Settings). At build/dev-server start, `vite.config.ts` parses the YAML and injects the map as a JSON string on `import.meta.env.VITE_TAB_VISIBILITY`. `App.tsx` reads it via `loadTabVisibility()`: a tab is shown unless its key is explicitly `false`. The same file also supplies the dev-server `host` / `port` and the backend proxy target.

#### 3.4 Reusable components

| Component | Role |
|---|---|
| `ConfigManager` | Generic CRUD manager for a "resource kind" (Device, Profile). Renders the header toolbar (Refresh / optional extra actions / Add), an optional search box, a Name+Actions table, and the JSON editor modal. Add/Edit/Delete/View are wired to a supplied `CrudApi`. |
| `JsonView` | Zero-dependency read-only JSON syntax highlighter (keys/strings/numbers/booleans/null colored) rendered into a dark |

| | |
|---|---|
| | `<pre>`. Exports `jsonModalWidth(text)` which sizes the modal to the widest line (clamped 520–1100px). |
| `JsonEditor` | Editable JSON field with live syntax highlighting (used for Add/Edit). |
| `markdown.css` | Styling for rendered Markdown ( `.chat-markdown` ) used by skills and the assistant answer. |
| ProfileFormEditor | Guided (form) profile editor: profile name / description / execution mode plus a collapsible card per task (case type, case id, TestRail case id, device, plan id, repeat, branches, timeout) with duplicate/remove and the **Auto-match** device button. It reads and writes the same JSON text as the JSON tab, normalizes defaults into it, and preserves unknown keys. |
| profileModel.ts | Plain helpers shared with the editor (types, `CASE_TYPES`, `EXEC_MODES`, `TASK_DEFAULTS`, `CONFIG_DEFAULTS`, `DROPPED_TASK_KEYS` / `DROPPED_CONFIG_KEYS`, parse/stringify/normalize). Kept out of the component file so React Fast Refresh keeps working. |

| | |
|---|---|
| SplitProfileButton | Profile row action: multi-select devices, call `/v1/profile/split`, and render the per-case assignment table (device, compatible devices, reason, fallback flag). |
| ConfigManager (tabs) | When a `formEditor` is supplied the Add/Edit/View dialog shows **Guided** and **JSON** tabs over one shared JSON string; the guided tab uses a fixed width while the JSON tab keeps auto-sizing, and the body scrolls inside the modal. |

#### 3.5 API client

`api/client.ts` centralizes all HTTP calls through base `/api` (Vite proxies to `127.0.0.1:9888`, stripping `/api`). It surfaces FastAPI `detail` on errors. List helpers strip file suffixes (device/profile `.json`, skill `.md`) so names round-trip cleanly.

### 4. Backend Design

#### 4.1 Application & configuration

`app.py` builds the FastAPI app via `create_app()`, mounts the aggregate `api_router`, and adds a fully-open `CORSMiddleware` (all origins/methods/headers). It runs under Uvicorn bound to `config.host:config.port` (default `0.0.0.0:9888`).

`conf/config.py` exposes a frozen `Config` dataclass: `host, port, base_url, api_key, key_id, key_alias, model, timeout_sec, max_tokens, temperature`. Each field is resolved by precedence **environment variable → YAML → error**. YAML has two sections — `server_config` (host/port) and `agent_config` (AI gateway settings) — plus an `agent_model` list of selectable models. Environment overrides use the `QATOOLS_*` prefix; `load()` raises `ConfigError` listing all missing keys.

#### 4.2 Router wiring & path constants

`controllers/__init__.py` sets up `sys.path` (adds the QATools root, backend root, and `4_runcase/common`), defines all filesystem path constants, loads the saved TestRail plans, configures the shared logger (console-only), then imports the nine sub-controllers and mounts their routers. Key path constants:

| Constant | Location | Holds |
|---|---|---|
| `device_conf_dir` | `config/device_conf` | device JSON files |
| `profile_conf_dir` | `4_runcase/profiles` | profile JSON files |
| `runcase_dir` | `4_runcase` | `test_runner.py`, cache & report files |
| `roku_test_dir` | `99_result_history/roku_test` | roku_test logs |
| `automated_tests_dir` | `automated_tests` (repo) | git branch operations |
| `testrail_data_dir` | `data/testrail_data` | cached TestRail case JSON |
| `testrail_conf_path` | `backend/conf/testrail.yml` | saved plans |
| `user_conf_path` | `backend/conf/user.yml` | user list |
| `skill_conf_dir` | `backend/skill_data` | skill folders (`<name>/SKILL.md`) |

⚠ **Import ordering.** All module-level names (path constants, `testrail_conf`, `logger`) are defined *before* the sub-controllers are imported, because each controller does `from . import logger, <dir>` at import time. Importing controllers first would raise `ImportError`.

#### 4.3 Host / port & frontend configuration

Zen has two independently-configured processes — the backend (FastAPI, port 9888) and the frontend dev server (Vite, port 9898) — each with its own config file. This subsection consolidates all host/port and tab-visibility settings.

##### 4.3.1 Backend host & port

The backend bind address comes from the `server_config` section of `backend/conf/config.yml`, resolved by `conf/config.py` (env var → YAML → error). Uvicorn binds to `config.host:config.port`.

| Setting | YAML (config.yml) | Env override | Default |
|---|---|---|---|
| Backend host | `server_config.host` | `QATOOLS_SERVER_HOST` | `0.0.0.0` |
| Backend port | `server_config.port` | `QATOOLS_SERVER_PORT` | `9888` |

**backend/conf/config.yml (excerpt)**

```yaml

server_config:
  host: "0.0.0.0"
  port: 9888

```

#### 4.3.2 Frontend host / port & backend proxy target

The frontend is configured entirely by `frontend/conf/config.yml`, read at build/dev-server start by `vite.config.ts`. It sets the dev-server bind (`server`) and the backend the SPA proxies `/api/*` to (`backend`).

| Setting | YAML key | Default | Effect |
|---|---|---|---|
| Frontend host | `server.host` | `0.0.0.0` | Vite dev-server bind address. |
| Frontend port | `server.port` | `9898` | Vite dev-server port. |
| Backend host | `backend.host` | `127.0.0.1` | Target host for the `/api` proxy. |
| Backend port | `backend.port` | `9888` | Target port for the `/api` proxy. |

**frontend/conf/config.yml (excerpt)**

```yaml

server:                 # Vite dev server bind
  host: "0.0.0.0"
  port: 9898
```

```yaml

backend:                # target of the /api proxy
  host: "127.0.0.1"
  port: 9888

```

`vite.config.ts` parses this file with a small indent-aware reader (no YAML dependency) and applies `server.host` / `server.port` to the dev server and `http://backend.host:backend.port` as the `/api` proxy target (the `/api` prefix is stripped on the way through). Changes take effect on the next dev-server restart.

#### 4.3.3 Left-nav tab visibility & order

Which tabs appear in the left navigation is controlled by the `tabs:` map in `frontend/conf/config.yml`. Each key maps to a boolean; a tab is shown unless its key is explicitly `false`.

**frontend/conf/config.yml — tabs section**

```yaml

tabs:
  device:   true
  profile:  true
  runcase:  true
  rokutest: true
  testrail: true
  agent:    true      # the "Prompt" tab

```

| Key | Tab label |
|---|---|
| `device` | Device |
| `profile` | Profile |
| `runcase` | Run Case |
| `rokutest` | Roku Test |
| `testrail` | TestRail Case |
| `agent` | Prompt |

**Mechanism.** At build/dev-server start, `vite.config.ts` reads the `tabs` map and injects it as a JSON string on `import.meta.env.VITE_TAB_VISIBILITY` (via Vite `define`). At runtime `App.tsx` calls `loadTabVisibility()` to parse it: the left menu renders only visible tabs, the initial page is the first visible tab, and hidden pages are never mounted. Setting a key to `false` hides both the menu item and its page. Changes require a dev-server restart (the value is injected at config-load time, not at runtime).

Example — a read-only deployment that only exposes TestRail browsing and the AI assistant: set `device/profile/runcase/rokutest` to `false`, leave `testrail/agent` as `true`, and restart the frontend.

### 5. HTTP API Reference

All routes are under `/v1`. 175 endpoints across nineteen controllers (Dashboard, Donut, Activate and Activate Account added in §5.18–5.21).

#### 5.1 Device — `/v1/device`

| Method | Path | Purpose | Params / Body | Returns |
|---|---|---|---|---|
| GET | `/list` | list device files | — | `{devices: [...]}` |
| GET | `/{device_name}` | view one device | path name | device JSON |
| POST | `/add/{device_name}` | create (409 if exists) | body: JSON | `{message}` |
| PATCH | `/update/{device_name}` | overwrite (must exist) | body: JSON | `{message}` |
| DELETE | `/delete/{device_name}` | delete | path name | `{message}` |
| GET | `/list_details` | list devices with presence flags per sub-section (QA Bot, Maxwell, Power, AVR, HDMI 1-3, STB) + IP | — | `{devices: [{name,ip_address,...}]}` |

| | | | | |
|---|---|---|---|---|
| GET | `/device_info/{device_name}` | live Roku device info (Account Type, Linked, 4K, tuner types, …) | path name | `{device_info}` |
| GET | `/download/{device_name}` | download the config as a .json attachment | path name | file |
| PATCH | `/sync_with_qa_hub/{device_name}` | refresh the local config from QA Hub | path name | `{Message}` |
| PATCH | `/reservations/{device_name}` | reserve the device on QA Hub for 30 days (steal) | path name | `{message}` |

Stored as `<device_conf_dir>/<name>.json`. `/list_details` feeds the Device table's capability columns, and the same per-device summary (peripherals + live Roku info) is what Profile **Split** and **Auto-match** reason over (§5.2).

#### 5.2 Profile — `/v1/profile`

| Method | Path | Purpose | Params / Body | Returns |
|---|---|---|---|---|
| GET | `/list` | list profiles | — | `{profiles:[...]}` |
| GET | `/generate_with_testrail_data` | build a profile from a cached TestRail data file | query: `plan`, `device`, `testrail_data_file` | `{message}` |

| | | | | |
|---|---|---|---|---|
| GET | `/{profile_name}` | view one profile | path name | profile JSON |
| POST | `/add/{profile_name}` | create | body: JSON | `{message}` |
| PATCH | `/update/{profile_name}` | overwrite | body: JSON | `{message}` |
| DELETE | `/delete/{profile_name}` | delete | path name | `{message}` |
| GET | `/list_details` | profiles with summary fields (description, total_tasks, execution_mode) | — | `{profiles:[...]}` |
| POST | `/split/{profile_name}` | AI-split the profile's cases across selected devices | `{devices:[...]}` | `{message, profiles:[...], assignments:[...]}` |
| POST | `/match_device` | pick the device best matching one TestRail case | `testrail_case_id` | `{device, reason}` |

⚠ Route ordering matters: the static `/generate_with_testrail_data` route is declared *before* the dynamic `/{profile_name}` route so it is not shadowed. `plan` is validated as an integer, and `testrail_data_file` is guarded against path traversal.

**Profile generation logic.** Reads `<testrail_data_dir>/<file>`; template `{name:"<device>_<plan>", description, tasks:[], execution_mode:"SERIAL"}`. For each case that has an `automation_id`, it appends a task whose config sets `case_type` = `BDD` when the id starts with `bdd-` (else `UT`), `case_id` = the id (BDD ids are split on `-`), plus `device`, `testrail_plan_id`, `repeat:0`, and `main` branches. Cases without an automation id are skipped. The result is saved as `profile_<base>_<device>.json`, each task using the single-`config` format of §9.2 with `timeout: 360000`.

**Split & Auto-match (AI case → device matching).** Two endpoints share one pipeline:

| Endpoint | Scope | Candidate devices | Result |
|---|---|---|---|
| `POST /v1/profile/split/{profile_name}` | every task of a profile | the devices in the request body | N new profiles + an assignment table |
| `POST /v1/profile/match_device` | one TestRail case | **all** device files | `{device, reason}` for the Guided editor's Auto-match button |

##### 5.2.1 Shared pipeline

1. **Device fact sheets** — `_device_summary()` runs per device in a thread and returns a credential-free summary: `peripherals` (`QA Bot`, `Maxwell`, `STB` = any `devices_config` entry with `device_type == "ROKU"`, `Power`, `AVR`, `HDMI 1-3`, `Tuner input`) from the device JSON, plus the `_ROKU_INFO_KEYS` subset of live Roku info via `_load_roku_info()` (`_DEVICE_INFO_TIMEOUT = 8` s; a failure is logged and the device is still matched on its peripherals).
2. **Case conditions** — `_case_conditions()` pulls each case from TestRail: text fields `custom_preconds`, `custom_testobjective`, `custom_steps` (truncated to `_CASE_TEXT_LIMIT = 900` chars) and label fields `custom_mode_account_status`, `custom_region`, `custom_product_area`, `custom_functionality`. Dropdown ids are resolved to labels by `_load_case_field_labels()`, which calls `get_case_fields` once and caches it (a failed fetch is deliberately *not* cached).

3. **AI verdicts** — cases are chunked into `_AI_BATCH_SIZE = 10` and sent with at most `_AI_CONCURRENCY = 3` in flight (TestRail reads use `_TESTRAIL_CONCURRENCY = 8`, each with `_TESTRAIL_TIMEOUT = 8` s). The prompt is the configurable Split template (§5.12). The model must answer only `{"cases":[{"key","compatible":[…],"reason"}]}`; `_extract_json()` strips code fences / prose and `_map_verdicts()` maps replies back by key, TestRail id, bare case id, or — as a last resort — positionally.
4. **Balancing** (Split only) — `_balance()` walks cases from most to least constrained and gives each one to the least-loaded compatible device, tie-broken by selection order. Cases with an empty compatible list go to `device_names[0]`, i.e. the **first selected device**.
5. **Persistence** (Split only) — `_write_new_split_profile()` writes `<profile>__001.json`, `__002`, … with `open(path, "x")`. The exclusive-create mode means a name taken by an existing (or concurrently created) profile is skipped rather than overwritten. `_strip_dropped_task_keys()` removes the unmanaged keys on the way out, and each copied task gets `config.device` rewritten to its target.

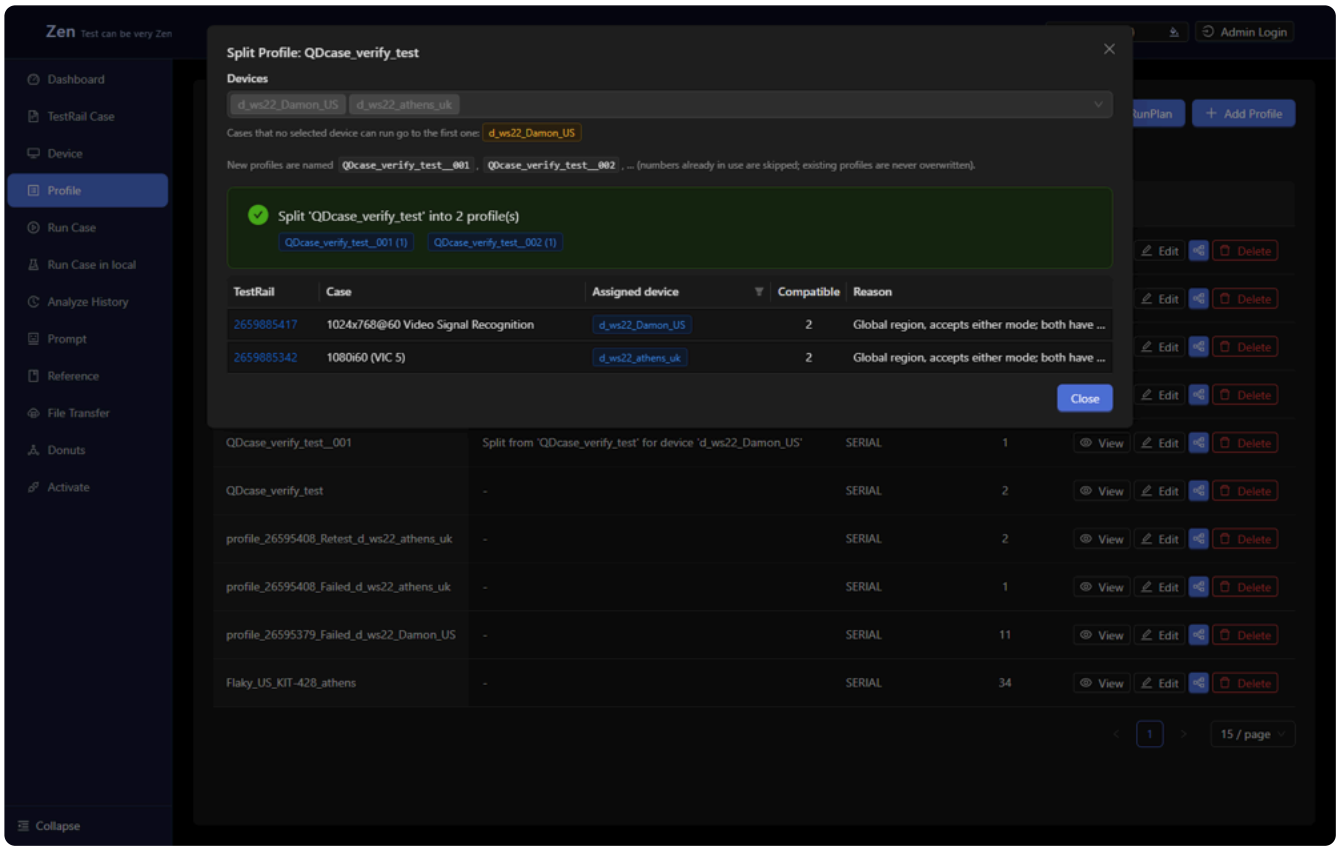
#### 5.2.2 Failure behaviour

| Condition | Split | Auto-match |
|---|---|---|
| no devices / profile has no tasks | HTTP 400 | HTTP 400 if the id is not all digits; 400 if no devices exist |
| TestRail case 404 | case keeps a `note` and is still matched | HTTP 404 |
| TestRail unreachable | degrades to *"matching without case fields"* — the device side alone decides | |
| one AI batch fails | its cases are flagged `ai_failed` and treated as compatible with every device | n/a (single batch) |
| all AI calls fail | HTTP 502 | HTTP 502 |
| no compatible device | case falls back to the first selected device (`fallback: true`) | HTTP 404 carrying the model's reason |

#### 5.2.3 Split response


```
{
  "message": "Split 'QDcase_verify_test' into 2 profile(s)",
  "profiles": [
    {"device": "d_ws22_Damon_US",  "profile": "QDcase_verify_test__001", "count": 1},
    {"device": "d_ws22_athens_uk", "profile": "QDcase_verify_test__002", "count": 1}
  ],
  "assignments": [
    {
      "case_id": "2659885417",
      "testrail_case_id": 2659885417,
      "title": "1024x768@60 Video Signal Recognition",
      "device": "d_ws22_Damon_US",
      "compatible": ["d_ws22_Damon_US", "d_ws22_athens_uk"],
      "reason": "Global region, accepts either mode; both have HDMI inputs.",
      "fallback": false,
      "ai_failed": false
    }
  ]
}
```


The UI renders `assignments` directly as the result table:




#### 5.2.4 Sequence — Split

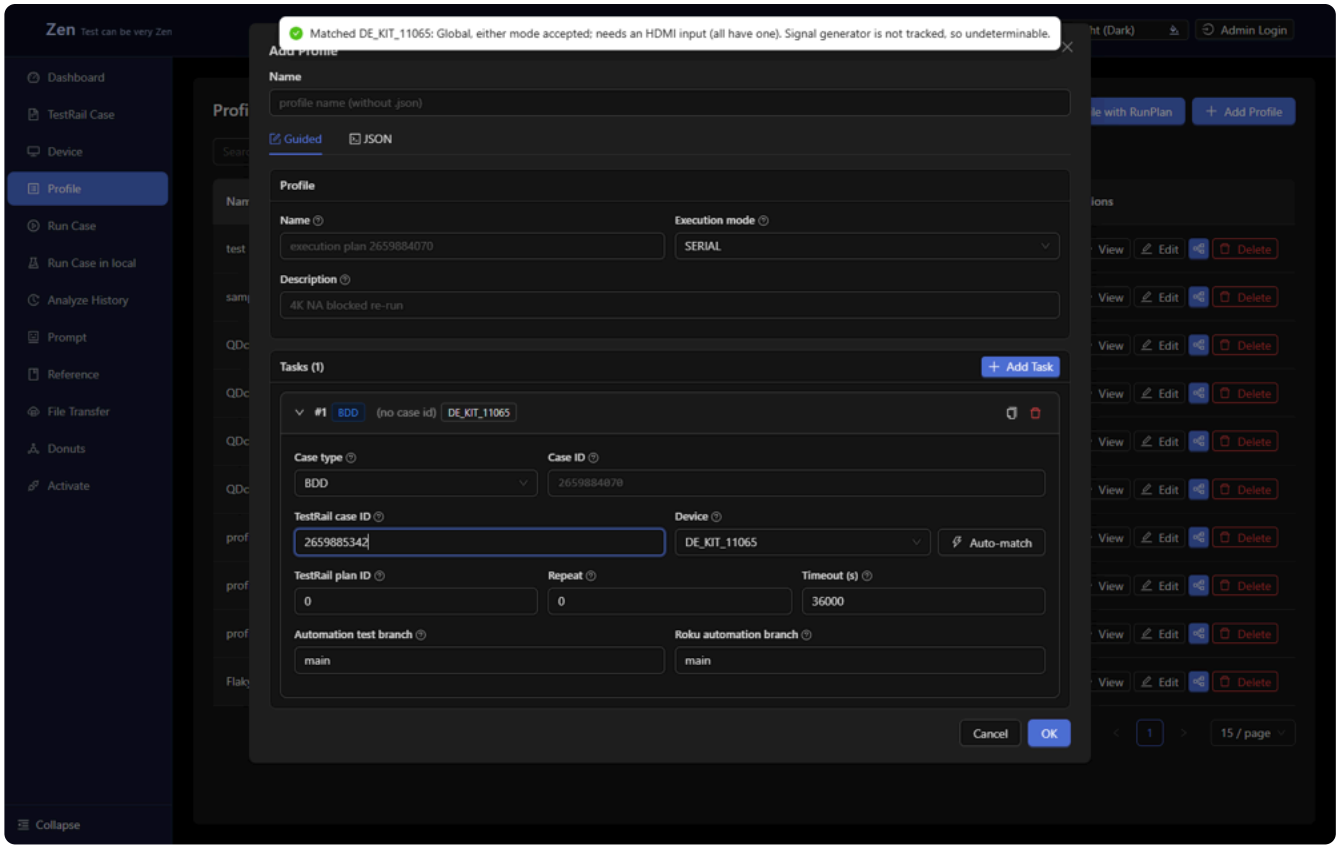



#### 5.2.5 Sequence — Auto-match

Auto-match is the single-case form of the same pipeline. It skips balancing and persistence entirely and simply returns the **first** compatible device, so repeated calls are deterministic and do *not* spread load.




The matched device is written straight into the shared profile JSON by the Guided editor:




⚠ The response's `device` carries a `.json` suffix (`"DE_KIT_11065.json"`); the frontend strips it before writing `config.device`.

### 5.3 Run Case — `/v1/runcase`

| Method | Path | Purpose | Params | Returns |
|---|---|---|---|---|
| POST | `/run` | spawn `test_runner.py -p <profile> -s <select>` detached | `profile_name`, `select`=All | `{message, pid}` |
| GET | `/list_running` | running runner processes (ps-parsed) | — | `{running_cases}` |
| GET | `/status` | same ps-parse | — | `{status}` |

| | | | | |
|---|---|---|---|---|
| POST | `/stop` | `kill -9 <pid>` | `pid` | `{message}` |
| GET | `/list_cache` | list `*.cache` | — | `{cache_files}` |
| GET | `/view_cache/{cache_file}` | read cache file | path | `{cache_file_content}` |
| GET | `/delete_cache/{cache_file}` | delete cache file | path | `{message}` |
| GET | `/list_report` | list `*summary_report.txt` | — | `{summary_report_files}` |
| GET | `/view_report/{report_file}` | read report (404 if missing) | path | `{summary_report_content}` |
| GET | `/delete_report/{report_file}` | delete report | path | `{message}` |
| GET | `/clear` | remove all cache + report files | — | `{message}` |
| GET | `/list_suites` | QA Hub suites for a user | `user`, `state`=running | `{suites: [...]}` |
| GET | `/terminate_suite` | QA Hub terminate | `suite_id` | `{status}` |
| GET | `/suite_build_url` | resolve Jenkins console URL | `suite_id` | `{build_number, url}` |

| | | | | |
|---|---|---|---|---|
| | | from suite build number | | |
| POST | `/analyze_profile` | AI-analyze the non-passing cases of a profile's latest run | `profile_name`, `locale` | `{profile_report, status, count_case, count_nopass_case, case_analyze_report}` |
| GET | `/log` | tail `4_runcase/logs/runcase.log` | `lines`=500 | `{log}` |
| GET | `/user_query_list` | saved user names for the Running Suites search box | — | `{users: [...]}` |
| POST | `/user_query_list/add` | remember a user name | `{name}` | `{message}` |
| DELETE | `/user_query_list/{name}` | forget a user name | path name | `{message}` |

The controller dynamically loads `4_runcase/test_runner.py`. `/run` launches `python3 -u test_runner.py` in a new session with `cwd=runcase_dir`, returning its PID. Running-process discovery parses `ps aux` for `test_runner.py` and extracts the `-p` / `-s` args.

**Auto analyze.** `POST /v1/runcase/run` also accepts `auto_analyze` (bool) and `locale`. When enabled, a daemon thread waits for the runner process to exit, finds the newest `*<profile>*_summary_report.txt` written after the run started, and calls

`agent_analyze` for every **non-passing** task (statuses outside `pass/passed/success/successful`), using that task's `log_url` as the prompt. `POST /v1/runcase/analyze_profile` runs the exact same `auto_analyze_profile()` on demand for the latest run of a profile.

- **Reports.** Each case yields a Markdown file in `data/analyzer_history/`; a roll-up `p__<profile>__<nopass>__<total>__<locale>__<datetime>.yml` is written *immediately* with `status: analyzing`, rewritten after every finished case, and always flipped to `status: completed` in a `finally` block — so the UI can show live "Analyzing n/m" progress and never gets stuck. Writes are atomic (`.tmp` + `os.replace`).
- **Event loop.** The AI client is one shared `httpx.AsyncClient`, and an httpx pool may only be driven by a single loop. All analysis coroutines are therefore funnelled onto one dedicated, never-closed loop (`_analyze_loop()`), or the caller's running loop when supplied. `_run_coro_with_retry()` retries once on the "event loop is closed / bound to a different event loop" family of errors, which otherwise makes case 1 look fine and cases 2..n all fail. `run_case` is declared `async` precisely so the app loop exists and can be handed to the worker thread.
- A failing case is logged and skipped; the batch always completes.

#### 5.4 Roku Test — `/v1/rokutest`

| Method | Path | Purpose | Params | Returns |
|---|---|---|---|---|
| POST | `/run` | run `roku_test` CLI, log to timestamped file | `device`, `case_id`, `repeat`=0 | `{message, command, log_file, pid}` |
| GET | `/status/{pid}` | check process alive | path pid | `{pid, status, command?}` |
| GET | `/log/list` | list logs with state + parsed result | — | `{logs:[{name,state,mtime,result}]}` |

| | | | | |
|---|---|---|---|---|
| GET | `/log/view/{log_file}` | read a log (404 if missing) | path | `{log}` |
| GET | `/log/delete/{log_file}` | delete a log | path | `{message}` |
| GET | `/jobs` | tracked runs (newest first) with live running state | `limit` =50 | `{jobs: [...]}` |
| DELETE | `/jobs` / `/jobs/{job_id}` | clear finished jobs / forget one job | path job id | `{message}` |
| POST | `/jobs/{job_id}/stop` | stop one tracked run | path job id | `{message}` |
| GET | `/cache/{field}` | input history for `automation_id` / `testrail_case_id` | path field | `{values: [...]}` |
| POST | `/cache/{field}/add` | remember an input value | `{value}` | `{message}` |
| DELETE | `/cache/{field}/{value}` | forget an input value | path | `{message}` |
| GET | `/log/download/{log_file}` | download a log as an attachment | path | file |

**Result parsing** scans the log: `res: Error` → Fail, otherwise the last `res: Pass/Fail/Skipped/Blocked`, else Unknown. **State** is "running" when the log name appears in the process table (`ps -eo args | grep roku_test`).

#### 5.5 Branch — `/v1/branch/automated_tests`

| Method | Path | Purpose | Body | Returns |
|---|---|---|---|---|
| GET | `/list` | `git branch` | — | `{automated_tests_branches: [...]}` |
| GET | `/current` | current branch | — | `{automated_tests_branch}` |
| POST | `/checkout` | `git checkout` (shlex-quoted) | `{branch_name}` | `{checkout_result}` |

Operates local git in `automated_tests_dir`.

#### 5.6 User — `/v1/user`

| Method | Path | Purpose | Returns |
|---|---|---|---|
| GET | `/list` | read `user.yml` | `{users: {...}}` |

#### 5.7 TestRail — `/v1/testrail`

| Method | Path | Purpose | Params / Body | Returns |
|---|---|---|---|---|
| GET | `/status/list` | live instance statuses | — | `{statuses: [{id,name,label}]}` |

| | | | | |
|---|---|---|---|---|
| GET | `/plan/case/list` | cases of a status under a plan (cached) | `plan`, `status`=Failed, `sync`=false | `{plan,status,count,cases:[...],cached}` |
| GET | `/plan/list` | saved plans | — | `{plans:[...]}` |
| POST | `/plan/add` | add a plan, persist YAML | body: plan | `{success}` |
| DELETE | `/plan/delete` | remove a plan by id | `plan_id` | `{success}` |
| GET | `/testrail_data/list` | cached files for a plan (by status) | `plan` | `{<status>:"<file>"}` |
| GET | `/testrail_data/available` | all cached files grouped by plan→status | — | `{plans:{<plan>:{<status>:"<file>"}}}` |
| POST | `/plan/case/add` | add cases to a TestRail plan entry | body: plan/case ids | `{success}` |

Each case record is `{run_id, test_id, case_id, title, automation_id, link}`. Status ids/labels are fetched live (they are instance-specific) rather than hardcoded.

#### 5.8 Skill — `/v1/skill`

| Method | Path | Purpose | Body | Returns |
|---|---|---|---|---|
| GET | `/list` | skill names | — | `{skills:[...]}` |

| | | | | |
|---|---|---|---|---|
| GET | `/{skill_name}` | read SKILL.md | — | `{skill}` |
| POST | `/add/{skill_name}` | create (400 if exists) | `{skill}` | `{message}` |
| PUT | `/update/{skill_name}` | overwrite (404 if missing) | `{skill}` | `{message}` |
| DELETE | `/delete/{skill_name}` | delete | — | `{message}` |
| GET | `/export/all` | download the whole `skill_data` tree as a .zip | — | file |

**Storage layout.** Each skill is stored as a *directory* containing a single Markdown file:

backend/skill_data/

```
skill_data/
├── <Skill name>/
│   └── SKILL.md          # the skill content (Markdown)
├── my_jira_helper/
│   └── SKILL.md
└── release_checklist/
    └── SKILL.md
```

The skill name is the directory name; the content always lives in `<Skill name>/SKILL.md`. `GET /v1/skill/list` returns the directory names (only folders that contain a `SKILL.md`), and the item endpoints read/write that file. Deleting a skill removes both the file and its directory.

**Claude Code compatibility — bidirectional skills migration.** This `<name>/SKILL.md` layout is deliberately identical to the skill format used by **Claude Code** (and the Claude Agent skills ecosystem), where each skill is a folder with a `SKILL.md` entry file. Because the on-disk structure matches, skills move between the two systems by a plain directory copy — no conversion step:

- **Claude Code → Zen:** copy a skill folder (e.g. `~/.claude/skills/<name>/`) into `backend/skill_data/`; it appears in the Prompt tab's Skills list immediately.
- **Zen → Claude Code:** copy a folder out of `backend/skill_data/<name>/` into a Claude Code skills directory; Claude Code picks it up as a native skill.

Authoring in Zen's Skills editor and authoring in Claude Code therefore target the same artifact, so a skill written in one is reusable in the other with zero rework.

#### 5.9 Agent — `/v1/agent`

| Method | Path | Purpose | Body | Returns |
|---|---|---|---|---|
| GET | `/info` | active agent settings | — | `{model, base_url, key_alias, max_tokens, temperature}` |
| GET | `/models` | selectable models (from config) | — | `[{name, description, model, max_tokens, temperature}]` |
| POST | `/switch` | change model/params at runtime | `{model?, max_tokens?, temperature?}` | `{message, agent}` |
| POST | `/chat` | chat, optionally RAG-augmented | `{prompt, use_internal=true, sources?}` | `{response, prompt, model,` |

| | | | | |
|---|---|---|---|---|
| | | | | `response_time}` |
| POST | `/search` | semantic search internal data | `{query, sources?, limit=10}` | `{query, sources, count, results:[...]}` |

A singleton `AgentService(AIClient(config))` is built at import; if the config is incomplete, the agent routes return **503**. `/switch` requires at least one of model/max_tokens/temperature and validates ranges (max_tokens > 0, temperature 0–2).

#### 5.10 Session — `/v1/session`

Multi-turn chat sessions for the Prompt tab. A session is a JSON file (`sess_<YYYYMMDD_HHMMSS>_<4hex>.json`) under `AGENT_SESSION_PATH` holding `{id, title, created, updated, messages:[{role, content, ts}]}`.

- `GET /list` — session summaries (id, title, updated, count), newest first.
- `GET /{sid}` — the full session including all messages.
- `POST /create` — create an empty session and return it.
- `DELETE /delete/{sid}` — delete a session file.

Non-HTTP helpers used by the agent controller: `get_history` (replays the last 30 messages for model context) and `append_turn` (appends a user+assistant turn, auto-titling from the first user message, truncated to 40 chars).

#### 5.11 Reference — `/v1/reference`

A library of two kinds: `ref` (links) and `docs` (uploaded Markdown). The `ref` / `docs` lists live in `backend/conf/ref.yml`; uploaded docs are written under `DOCS_DATA_PATH` (`data/docs/`).

- `GET /list` — both lists (each item tagged with its index) plus categories.
- `POST /add` — append an item to the ref or docs list.
- `PUT /update` — overwrite the item at a given index.
- `DELETE /delete` — delete the item at an index (params kind, index).
- `POST /doc/upload` — save a Markdown file under docs/ and add/refresh its record (overwrites same name).
- `GET /doc/content` — return a Markdown doc's text (param name).

#### 5.12 Config — `/v1/config`

Reads/writes the YAML data stores under `backend/conf/` (`tabs_data.yml`, `agent_data.yml`, `analyze_data.yml`, … mapped by `config.yml`'s `data_files`) to drive the Settings tab. Tab, model, and Analyze changes apply live where possible; credential changes need a backend restart.

- `GET /tabs/list` — tabs with visibility + display order.
- `GET /tabs/{tab_name}/enable` / `GET /tabs/{tab_name}/disable` — show/hide a tab.
- `PUT /tabs/reorder` — rewrite the tabs map order (key order = display order).
- `GET /models/list`, `POST /models/add`, `PUT /models/update/{name}`, `DELETE /models/delete/{name}` — CRUD over the `agent_model` list.
- `GET /agent/get` / `PUT /agent/update` — agent credentials (secrets write-only/masked; empty values keep the stored secret).
- `GET /agent/default` / `PUT /agent/default` — the default model; the PUT persists to config.yml *and* applies to the live agent (no restart).
- `GET /analyze/get` / `PUT /analyze/update` — the Analyze `default_locale` + `prompt_template` (applied live; the shipped default locale is `en-US`).
- `GET /analyze/split_prompt/get` / `PUT /analyze/split_prompt/update` — the **Split Profile prompt** (`analyze_config.split_prompt_template` in `analyze_data.yml`). GET returns the configured template, the built-in `default_split_prompt_template`, `is_default` and the required `placeholders`. PUT rejects an empty template or one missing `{devices}` / `{cases}` with HTTP 400, and touches only that key. A configured template that loses a placeholder is ignored at split time in favour of the built-in default.
- `POST /admin/login` — verify admin credentials (bcrypt, from env.py `ADMIN_USERNAME` / `ADMIN_HASHED_PASSWORD`; default `admin` / `Zaq1@wsx`); gates the Settings tab. `GET /account/get` / `PUT /account/update` — read/write the editable `config/env.py` values — 4 masked secrets (`QAHUB_TOKEN`, `TESTRAIL_USERNAME`, `TESTRAIL_API_KEY`, `JIRA_TOKEN`) and 3 plain fields (`EMAIL`, `AUTOMATED_TESTS_RELATIVE_DIR`, `ROKU_AUTOMATION_RELATIVE_DIR`). `env.py` *is* the source of truth: the assignments are rewritten in place and applied after a backend restart. `POST /backend/restart` — restart the backend via `backend/restart.sh` (detached, so the response returns first).

#### 5.13 Analyze History — `/v1/analyze_history`

The AI log analyses saved by `POST /v1/agent/analyze`, stored as Markdown under `ANALYZER_PATH` (`data/analyzer_history/`) named `<testrail_case_id>__<profile>__<case_id>__<result>__<locale>__<datetime>.md`.

- `GET /list` — saved analyses with parsed name fields (testrail_case_id, profile, case_id, result, locale, date).
- `GET /categories` — distinct filter values.
- `GET /view` — a saved analysis's Markdown (param name).
- `DELETE /delete` — delete a saved analysis (param name).

**Profile roll-up reports.** Auto analyze (§5.3) additionally writes `p__<profile>__<nopass>__<total>__<locale>__<datetime>.yml` next to the case reports:

- `GET /profile/list` — roll-up reports, newest first, filterable by `profile` / `locale` / `date`. Each record merges the parsed file name with the file body (body wins) and exposes `status` (`analyzing` | `completed`), `count_nopass_case`, `count_case`, `analyzed_count`, `case_analyze_report` and a `cases` list of `{testrail_case_id, name}` for direct links. Reports written before the `status:` line existed are treated as completed.
- `GET /profile/view` — raw YAML plus the parsed record.
- `DELETE /profile/delete` — delete a roll-up report.

File names are validated to a bare base name with a `.md` (case report) or `.yml` / `.yaml` (profile report) suffix, so neither endpoint can traverse out of `ANALYZER_PATH`.

#### 5.14 Log Analysis — `POST /v1/agent/analyze`

Runs an AI root-cause analysis of a failed-test log and saves the result to Analyze History. Query params: `profile_name` (default `"default"`; `"local"` for Run-Case-in-local runs), `locale` (defaults to `analyze_config.default_locale` when omitted), `testrail_case_id`, `case_id`, `case_test_result`. Body is an `AgentChatRequest` (`prompt`).

- The `analyze_config` (default locale + prompt template) is read *fresh per request*, so Settings edits apply with no restart.
- If the prompt is a single-line http(s) URL, the log is fetched server-side (`_fetch_log_text`; the model cannot open links); a fetch failure returns HTTP 502.

- The template substitutes `{lang}` ("Chinese (简体中文)" when locale is zh-CN, else "English") and `{content}` (the log). An empty template returns HTTP 400 directing the user to Settings.
- The rendered analysis is saved to `data/analyzer_history/` and returned as `{response, prompt, model, response_time}`.

> **Device sync.** The Device controller also exposes `PATCH /v1/device/sync_with_qa_hub/{device_name}`, which refreshes a device config from QA Hub.
>
> **Data directories.** All runtime data now lives under `QATools/data/`, with path constants centralized in `config/env.py`: `data/runcase_report` (REPORT_PATH & CASE_CACHE_PATH), `data/testrail_data` (TESTRAIL_DATA_PATH), `data/analyzer_history` (ANALYZER_PATH), `data/agent_session` (AGENT_SESSION_PATH), `data/skill_data` (SKILL_DATA_PATH), and `data/docs` (DOCS_DATA_PATH).

#### 5.15 Cmd / Run Script — `/v1/cmd`

The Cmdline sub-tab of File Transfer: manage shell scripts/data files under `QATools/data/cmd` and run arbitrary command lines (and interactive SSH sessions) server-side. Run timeout 600s; output capped at 200k chars.

- `GET /tree`, `GET/POST /file`, `POST /upload`, `DELETE /file`, `POST /mkdir`, `DELETE /dir` — file/dir CRUD confined to the cmd dir (path-traversal guarded; `.sh/.py/.bash` chmod 755).
- `POST /run` — run a command line via the shell from the cmd dir; returns `{cmdline, exit_code, timed_out, elapsed, stdout, stderr, cwd}`. The final `$PWD` is echoed back so `cd` persists across calls; `ssh/scp` get host-key prompts suppressed and are wrapped with `sshpass -e` when a password is supplied (password via the `SSHPASS` env var, never in argv/logs). Each run is recorded in config.yml `file_transfer.cmd_cache`.
- `GET /cache` / `DELETE /cache` — the run history.
- `POST /ssh/start` — open a persistent interactive `ssh -tt user@host` on a PTY; returns a `session_id`. `POST /ssh/input` sends stdin, `GET /ssh/output?since=` streams new output (`{data, offset, alive}`), `POST /ssh/stop` closes it. Idle sessions reaped after 1800s; 500KB output buffer.

#### 5.16 File Transfer — `/v1/file_transfer`

The File Transfer sub-tab: a managed file root (default `data/file_transfer`, overridable in config.yml `file_transfer.root`), a built-in HTTP file server, and scp upload with real progress.

- `GET/PUT /config` — effective `{root, http_port}` and updates.
- `GET /tree`, `POST /upload` (base64), `DELETE /file`, `POST /mkdir`, `DELETE /dir` — file/dir operations under the root.
- `GET /http/status`, `POST /http/start`, `POST /http/stop` — manage a singleton `python -m http.server <port> -b 0.0.0.0 -d <root>` (PID tracked in `data/.file_transfer_http.pid`).
- `POST /scp/start` — background `sshpass + scp` upload (file or `-r` dir); returns a `job_id`. `GET /scp/status/{job_id}` reports real `percent` (polling remote `du -sb` vs. local size). On success the target is upserted into `file_transfer.scp_server_cache`.
- `GET /scp/servers` / `DELETE /scp/servers` — the remembered scp targets (`server/upload_dir/username`) powering the Host autocomplete.

#### 5.17 Admin & Account Settings

The **Settings** tab is gated behind **Admin Login** (`POST /v1/config/admin/login`, bcrypt; default `admin` / `Zaq1@wsx`). It exposes four areas: *Activate Account* (country → activation email, §5.21, applied live), *Tab Settings* (visibility + order via the config tab endpoints), *Agent Settings* (credentials, Analyze locale/template, model CRUD + default — the AI gateway config), and *Account Settings* (the env.py credentials/paths: QA Hub token, TestRail email/username/API key, JIRA token, data base dir, roku_test log dir, automated_tests dir). Secrets are masked; account changes persist to `config/account.json` and apply after a backend restart, while tab/model/analyze changes apply live.

#### 5.18 Dashboard — `/v1/dashboard`

Read-only aggregation for the Dashboard page; local reads only (no external HTTP). Every card builder is wrapped in `_safe()` so one failing widget returns `{error}` without failing the response.

| Method | Path | Purpose | Returns |
|---|---|---|---|
| GET | `/summary` | all cards in one snapshot | `{generated_at, cards: {device, profile,` |

| | | | |
|---|---|---|---|
| | | | `runcase`, `rokutest`, `analyzehistory`, `testrail`, `agent`, `reference`, `filetransfer`, `donuts`, `activate`}} |
| GET | `/card/{name}` | refresh one card | `{generated_at, card}` |

#### 5.19 Donut — `/v1/donut`

Thin proxy over the QA Hub v1 donut API (token header). Launched runs are recorded in `data/donut_runs.json` (max 200) and correlated to Jenkins jobs by serial + donut / `donut_config_id`, since `POST /donut/run` returns no suite id.

| Method | Path | Purpose | QA Hub call |
|---|---|---|---|
| GET | `/products`, `/branches/{product}` | catalogue | `donut/product`, `donut/branch/{p}` |
| GET | `/for_device/{device_name}` | donuts the device is registered on (serial from device_conf) | `donut/device` |
| GET | `/configuration` | `donut_config_id` for product/branch/donut | `donut/configuration/{p}/{b}/{d}` |
| POST | `/run` | trigger a run; 202 → queued | `POST donut/run` |

| | | | |
|---|---|---|---|
| | | ( `queued_run_id` , `busy_devices` ) | |
| GET | `/runs` | local history + live jobs / queued requests | `donut/jenkins-running` , `donut/queued-runs` |
| DELETE | `/runs/{run_id}` | remove a local record only | — |
| POST | `/abort/job` | stop a running Jenkins donut job | `DELETE donut/jenkins-running/{p}/{b}` |
| POST | `/abort/queued` | cancel a queued request | `DELETE donut/queued-runs/{p}/{b}` |
| POST | `/abort/suite/{suite_id}` | kill a suite (state → kill) | `PATCH suite/{id}` |
| GET | `/suite/{id}` , `/suites` , `/summary/{id}` | suite detail / list / pass-fail summary | `suite` , `suite/{id}/tests` |
| GET | `/results/{build_number}` | finished runs: pass %, stage, logs URL | `donut/results/{build}` |

#### 5.20 Activate — `/v1/activate`

Drives the `5_activate` scripts. Modes: `connected` ( `factory_reset_and_activate.py` ) and `unconnected` ( `factory_reset_and_unconnected_mode.py` ). Each device runs as a detached `setsid`

process with its own log in `5_activate/logs`; jobs are tracked in `backend/conf/activate_jobs.json`.

| Method | Path | Purpose | Body / Returns |
|---|---|---|---|
| POST | `/emails` | default activation email per device | `{device_names}` → `{emails}` |
| GET | `/email/{device_name}` | default activation email for one device | `{device_name, email}` |
| POST | `/precheck` | ping + `precheck_connection.py`; wireless → Unconnected is not allowed | `{device_names, mode}` → `{results[]}` |
| POST | `/start` | precheck then start all allowed devices in parallel | `{device_names, mode, skip_precheck, emails}` → `{started, rejected, prechecks}` |
| POST | `/connected/start`, `/unconnected/start` | single-device shortcuts | `{device_name}` → `{pid, log_file}` |
| GET / DELETE | `/jobs`, `/jobs/{job_id}` | list jobs (running + result) / remove a finished job | — |
| GET / POST | `/status/{pid}`, `/stop/{pid}` | process state / kill the process group | — |

| | | | |
|---|---|---|---|
| GET / DELETE | `/log/list`, `/log/view/{log_file}`, `/log/{log_file}` | list / tail (last 5000 lines) / delete logs | — |

#### 5.21 Activate Account — `/v1/activate_account`

CRUD for the country → activation-email map in `config/activate_account.yml` (section `activate_account`, special key `DEFAULT`). Used by `/v1/activate/emails`: the first country key contained in the device name wins, else `DEFAULT`.

| Method | Path | Purpose |
|---|---|---|
| GET | `/list` | all accounts `{accounts: [{country,email}]}` |
| GET | `/{country}` | one account |
| POST | `/add` | add `{country, email}` (409 if exists) |
| PUT | `/update/{country}` | change the email |
| DELETE | `/delete/{country}` | remove |

## 6. AI Assistant Design (RAG)

#### 6.1 Components

- **AgentService** — thin facade: `info / chat / search / switch`.
- **AIClient** — async httpx wrapper over the OpenAI-compatible AI gateway (`ai-hub-lite`, base `/v1`). Auth: `Bearer <api_key>` plus `X-Api-Key-Id` / `X-Api-Key-Alias`. MCP calls go to `<gateway_root>/mcp-rest/tools/call`. A temperature guard omits the `temperature` parameter for reasoning models that only accept `temperature=1` (Claude 4.x/5.x Opus/Sonnet, GPT-5.5/5.6/6, gpt-5-codex).

- **jira_client** — JIRA REST v3 `/search/jql` (v2 removed by Atlassian); Basic auth; returns compacted issue records; also provides a dated JQL cheat-sheet for the model.

#### 6.2 Chat data flow ( `use_internal=true` )

Zen answers using a two-path retrieval strategy:

1. **Structured JIRA (JQL) path — tried first.** The model is asked (with a dated cheat-sheet: today's date, last-week Mon–Sun boundaries, `currentUser()` semantics) to translate the question into a single JQL line, or return `NONE`. The output is cleaned and sanity-checked for a JQL operator. If a valid JQL is produced, `jira_client.search_jql()` runs it (up to 25 issues) and the results become the CONTEXT block.
2. **Semantic-search fallback.** If no JQL applies, Zen calls the MCP `ai_hub_search` tool over the selected sources (up to 6 docs, retried up to 3× for transient 5xx). Results are formatted into a numbered CONTEXT block (capped at 12,000 chars). If search fails or is empty, the CONTEXT explicitly instructs the model to say so — and *not* to claim it lacks internal access.
3. Zen always uses the RAG system prompt, sends `CONTEXT (Roku internal data): …` `QUESTION: …`, and returns the completion. With `use_internal=false` it does a plain completion.

```
chat(prompt, use_internal=true)
  ├─ _maybe_jql(prompt) --(model)--> "JQL" or NONE
  │       └─ if JQL: jira_client.search_jql() -> CONTEXT
  └─ else: ai_hub_search (MCP, retry x3) -> CONTEXT
          (empty/fail -> explicit "say so" note)
  -> _complete(RAG_SYSTEM_PROMPT, "CONTEXT ... QUESTION ...")
```

#### 6.3 Search data flow (MCP)

`search()` POSTs to `<gateway_root>/mcp-rest/tools/call` with `{name, server_id:"ai_hub_search", arguments:{query, limit, sources?}}`. The tool name is tried as `ai_hub_search-search` then `search`. Because the gateway intermittently returns 500 "Tool search not found", the whole attempt is retried up to **6×** with linear backoff. The MCP envelope ( `content[].text` JSON, or `structuredContent.result` ) is unwrapped into the results list; an `isError` envelope raises.

### 7. LLM Interface & Configuration

All LLM traffic goes through a single async client, `AIClient` ( `aiutil/ai_client.py` ), which speaks the **OpenAI-compatible** Chat Completions protocol to the Roku AI gateway **ai-hub-lite**. Semantic search uses the gateway's MCP endpoint. This section specifies the wire contract, authentication, parameter handling, error/retry behavior, and the full configuration reference.

#### 7.1 Endpoints

The gateway *base URL* is configured as `base_url` (e.g. `https://ai-hub-lite.msc.rokulabs.net/v1`). Two families of calls are made:

| Purpose | Method & URL | Protocol |
|---|---|---|
| Chat completion (all model calls) | `POST {base_url}/chat/completions` | OpenAI-compatible |
| Semantic search (RAG) | `POST {gateway_root}/mcp-rest/tools/call` | MCP tools/call |

`gateway_root` is `base_url` with a trailing `/v1` removed — the MCP endpoint lives at the gateway root, outside the versioned API. The client is a single reused `httpx.AsyncClient` created with `base_url`, the auth headers below, and a request `timeout` of `timeout_sec`.

#### 7.2 Authentication

Every request carries these headers (built once at client construction):

| Header | Value | Notes |
|---|---|---|
| `Authorization` | `Bearer <api_key>` | required |
| `Content-Type` | `application/json` | required |
| `X-Api-Key-Id` | `<key_id>` | sent only when configured |
| `X-Api-Key-Alias` | `<key_alias>` | sent only when configured; identifies the caller (e.g. `kzheng`) |

⚠ Secrets (`api_key`, `key_id`) are never returned by any Zen endpoint. `GET /v1/agent/info` exposes only `key_alias`.

#### 7.3 Chat completion request

`_complete(system_prompt, user_content)` builds a two-message conversation and posts it:

**POST {base_url}/chat/completions**

```
{
  "model": "claude-opus-5-5",
  "messages": [
    { "role": "system", "content": "<system prompt>" },
    { "role": "user",   "content": "<user content>" }
  ],
  "max_tokens": 128000,
  "temperature": 1          // OMITTED for reasoning models (see 7.4)
}
```

| Field | Source | Description |
|---|---|---|
| `model` | active agent setting | The currently selected model (changeable at runtime via `/v1/agent/switch`). |
| `messages` | caller | Always exactly one `system` + one `user` message. The system prompt differs by mode: default assistant, RAG assistant, or the JQL-translation prompt. |
| `max_tokens` | active setting or call override | Output cap. The JQL-translation call overrides this to `200`; normal chat uses the configured value. |
| `temperature` | active setting | Included only when the model supports a custom temperature (see 7.4). |

#### 7.4 Temperature handling for reasoning models

Some models (reasoning models) reject any `temperature` other than `1` and return HTTP 400. The client therefore **omits** the `temperature` field entirely for models whose id starts with a known reasoning-model prefix:

```

_TEMPERATURE_UNSUPPORTED_PREFIXES = (
    "claude-4.", "claude-opus-4-", "gpt-5-codex",
    "claude-5-", "claude-opus-5", "claude-sonnet-5",
    "gpt-5.5", "gpt-5.6", "gpt-6",
)

```

`AgentSettings.supports_temperature` checks the active model against these prefixes; when unsupported, the payload simply has no `temperature` key and the gateway applies the model default. This is why several entries in `agent_model` carry `temperature: 1` — it documents intent, but the value is not actually sent for those models.

#### 7.5 Chat completion response

Responses follow the OpenAI envelope. `_extract_message()` reads `choices[0].message.content`, tolerating either a plain string or a list of content parts (each with a `text` field), and returns the trimmed text. A non-JSON body, a missing `choices`, or a missing message raises `AIClientError`.

```

{ "choices": [ { "message": { "role": "assistant", "content": "<answer
text>" } } ] }

```

#### 7.6 MCP semantic-search call

Search posts an MCP `tools/call` envelope to `{gateway_root}/mcp-rest/tools/call`:

##### POST {gateway_root}/mcp-rest/tools/call

```

{
  "name": "ai_hub_search-search",   // primary; "search" tried as
fallback
  "server_id": "ai_hub_search",
  "arguments": { "query": "<text>", "limit": 10, "sources": ["JIRA"] }
}

```

The response is the MCP envelope: the results arrive as a JSON string inside `content[].text` (or `structuredContent.result`). `_extract_search_results()` unwraps it to a `results[]` list; an `isError:true` envelope raises. Each result is `{document_id, content, ...}`.

#### 7.7 Error handling & retries

| Call | Failure | Behavior |
|---|---|---|
| Chat completion | transport error / non-2xx / non-JSON | Raise `AIClientError` (surfaced to the controller as HTTP 502). No retry — a bad completion is not transient. |
| JQL translation | `AIClientError` | Swallowed → treated as "no JQL", falls through to semantic search. |
| MCP search | 500 "Tool search not found" (flaky gateway) | Retry the whole attempt up to **6×** with linear backoff ( `0.4 × attempt` s); try tool name `ai_hub_search-search` then `search` each round. A non-"not found" 4xx/5xx breaks early. |
| RAG search (from chat) | search fails after its own retries | Retried up to **3×** at the chat layer; if still failing, a CONTEXT note tells the model to say it could not reach internal data (never to claim it lacks access). |

#### 7.8 Runtime model switching

`POST /v1/agent/switch` mutates the in-memory `AgentSettings` (model / max_tokens / temperature) on the singleton client — no restart, no config-file write. At least one field is

required; `max_tokens` must be > 0 and `temperature` in 0–2. The switch is process-local and resets to the configured defaults when the backend restarts. `GET /v1/agent/models` lists the selectable models from the `agent_model` config list.

#### 7.9 Configuration reference

LLM settings come from `backend/conf/config.yml`, resolved by `conf/config.py` with precedence **environment variable → YAML → error**. The YAML path can be overridden with `QATOOLS_AGENT_CONFIG_PATH`. Missing required keys make `load()` raise `ConfigError` listing every missing key (and the agent routes then return 503).

**config.yml — `agent_config` section** (the active agent):

| Key | Env override | Type | Example | Meaning |
|---|---|---|---|---|
| `base_url` | `QATOOLS_AGENT_BASE_URL` | str | `https://ai-hub-lite.msc.rokulabs.net/v1` | Gateway base (chat at `/chat/completions`; MCP at root). |
| `api_key` | `QATOOLS_AGENT_API_KEY` | str | *(secret)* | Bearer token. |
| `key_id` | `QATOOLS_AGENT_KEY_ID` | str | *(secret)* | `X-Api-Key-Id` header. |
| `key_alias` | `QATOOLS_AGENT_KEY_ALIAS` | str | `kzheng` | `X-Api-Key-Alias` header. |
| `model` | `QATOOLS_AGENT_MODEL` | str | `claude-opus-5-5` | Default active model at startup. |
| `timeout_sec` | `QATOOLS_AGENT_TIMEOUT_SEC` | float | `300.0` | Per-request HTTP timeout. |
| `max_tokens` | `QATOOLS_AGENT_MAX_` | int | `128000` | Default output cap. |

| | | | | |
|---|---|---|---|---|
| | `TOKENS` | | | |
| `temperature` | `QATOOLS_AGENT_TEMPERATURE` | float | `1` | Default sampling temperature (omitted for reasoning models). |

**config.yml — `server_config` section**: `host` (`QATOOLS_SERVER_HOST`, default `0.0.0.0`) and `port` (`QATOOLS_SERVER_PORT`, default `9888`).

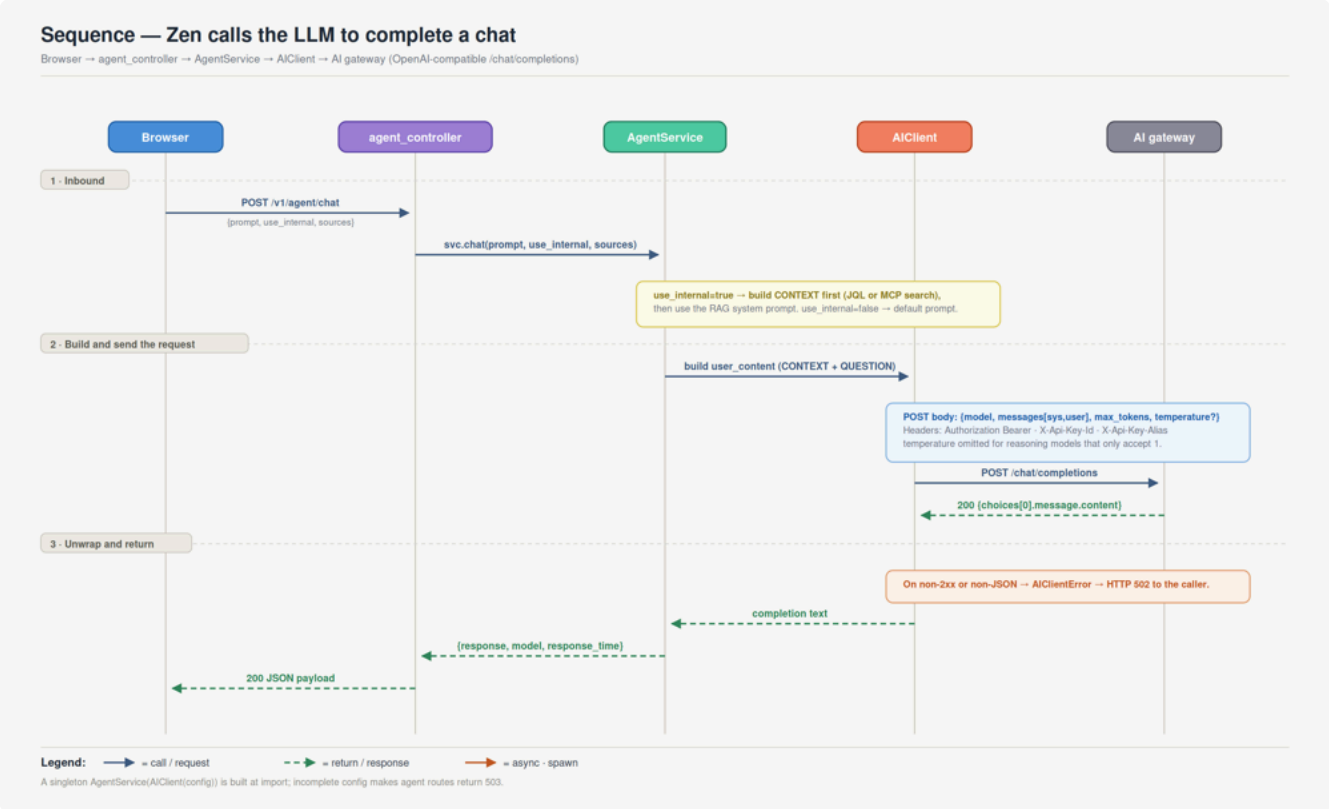
**config.yml — `agent_model` list**: the models offered by `GET /v1/agent/models` and the Prompt tab's Switch Model dropdown. Each entry is `{name, description, model, max_tokens, temperature}`. Currently configured (6 models — the live list; manage them in Settings → Agent Models):

| model | max_tokens | temperature | temperature sent? |
|---|---|---|---|
| `claude-opus-5-5` (default) | 128000 | 1 | no (reasoning) |
| `claude-4.7-opus` | 128000 | 0.7 | no (matches `claude-4.`) |
| `claude-5-sonnet` | 128000 | 0.7 | no (matches `claude-5-`) |
| `gpt-5.4` | 128000 | 0.7 | yes |
| `claude-5-opus` | 128000 | 1 | no (reasoning) |
| `claude-4.6-opus-context-1m` | 128000 | 0.7 | no (matches `claude-4.`) |
| `gpt-5.5` | 128000 | 1 | no (reasoning) |
| `gpt-6-luna` | 128000 | 1 | no (reasoning) |

To add a model, append an entry to `agent_model` and restart the backend. If the model is a reasoning model that only accepts `temperature=1`, ensure its id matches a prefix in `_TEMPERATURE_UNSUPPORTED_PREFIXES` (or add one) so the client omits `temperature` and avoids a 400.

#### 7.10 Chat call sequence

End-to-end view of a single chat request calling the LLM: from the browser through the controller, service, and `AIClient`, out to the AI gateway's `/chat/completions`, and back. (The RAG retrieval that builds CONTEXT before this call is detailed in §6.2 and §8.1.)

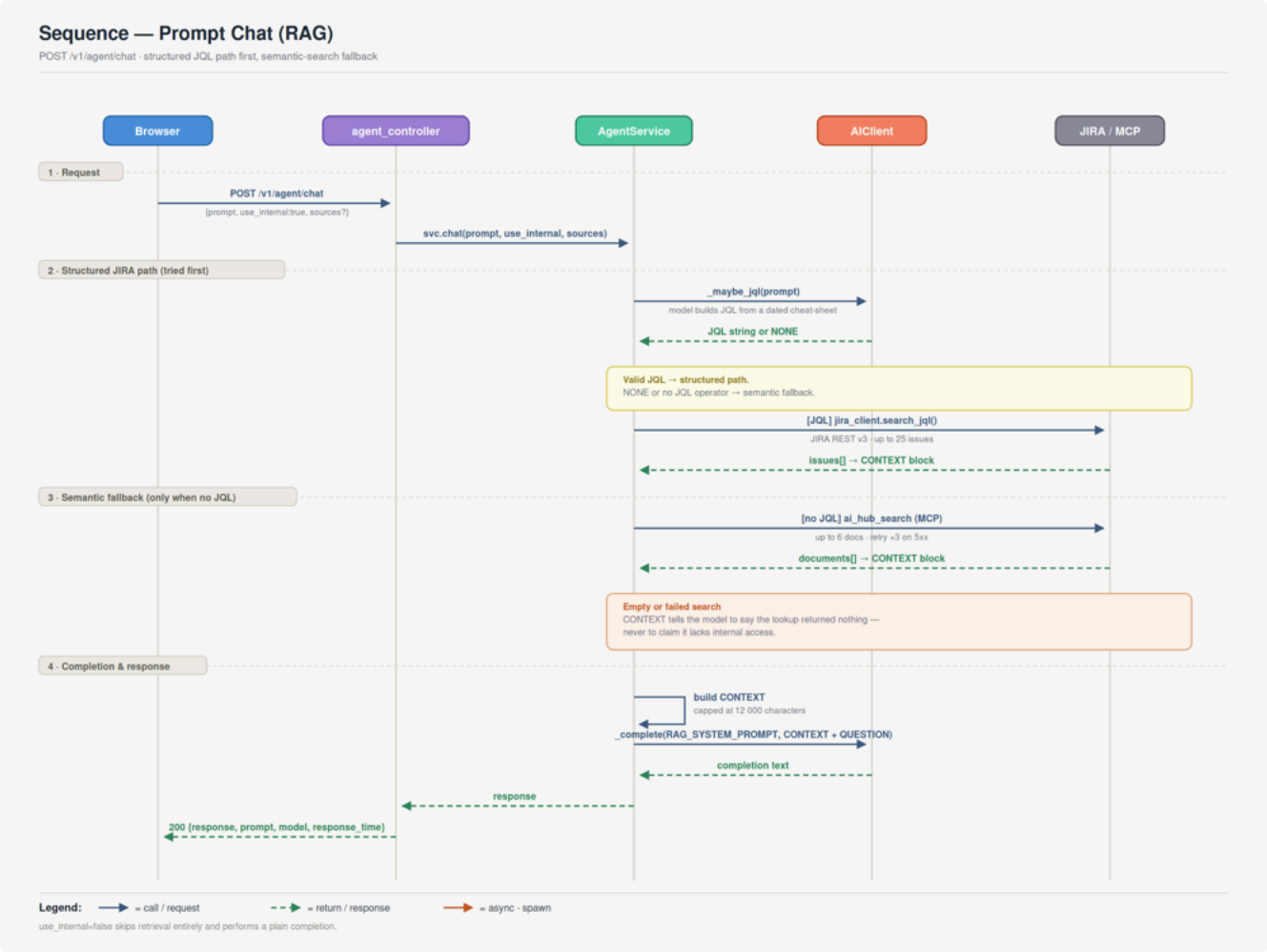



## 8. Sequence Diagrams

The key runtime flows, as message sequences. Solid arrows are calls; dashed green arrows are returns; grey arrows are outbound HTTP to external systems.

#### 8.1 Prompt Chat (RAG)

A chat request with `use_internal=true` follows a two-path retrieval strategy. Zen first asks the model to translate the question into JQL; if a valid JQL is produced it runs it against JIRA (structured path), otherwise it falls back to MCP semantic search. Either way the retrieved documents become a CONTEXT block that is prepended to the question for the final completion.



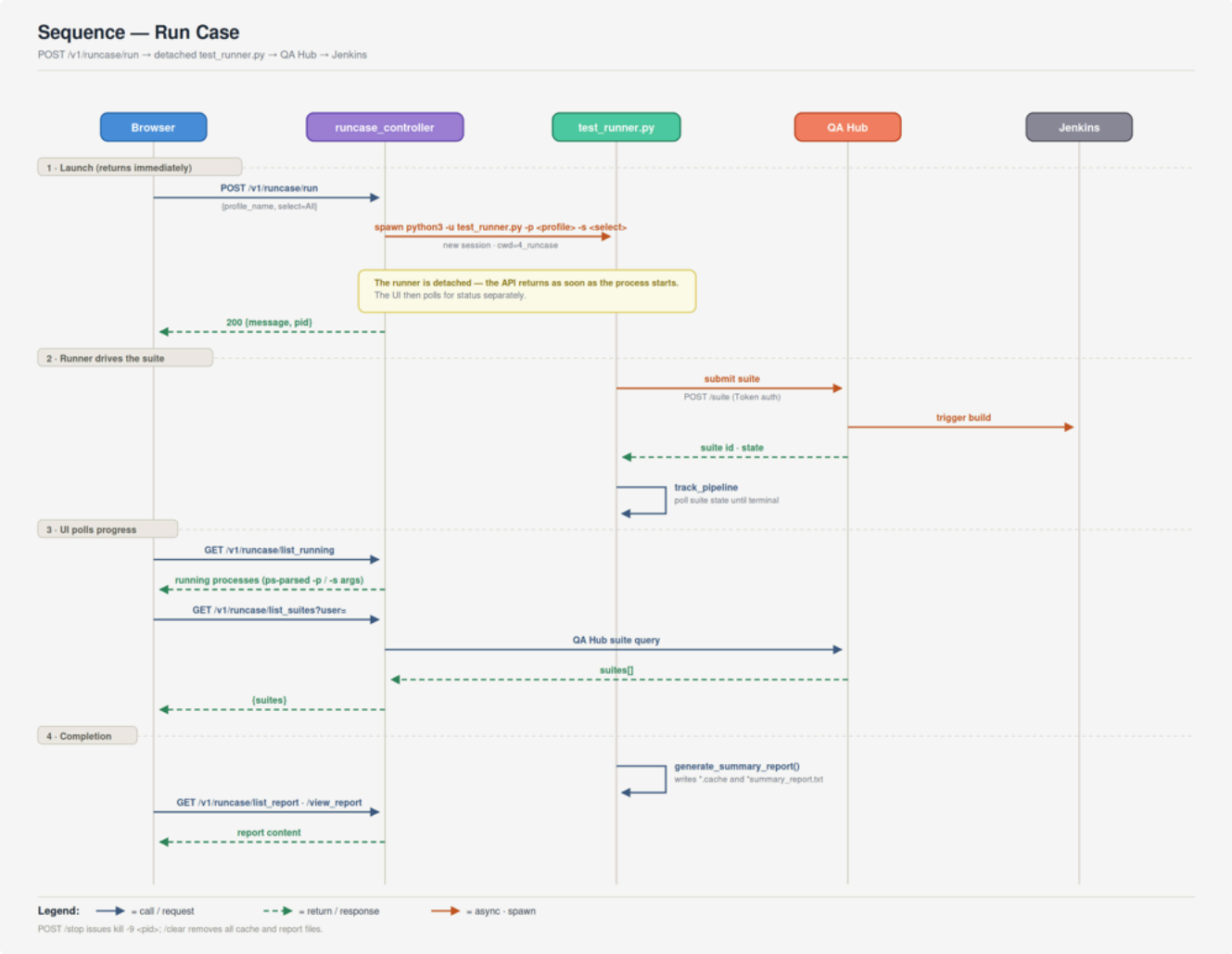

<details>
<summary>Step-by-step</summary>

1. Browser → `agent_controller`: `POST /v1/agent/chat` with the prompt and `use_internal:true`.
2. Controller → `AIClient.chat()`.
3. `AIClient` → AI gateway: `_maybe_jql()` asks the model (with a dated JQL cheat-sheet) to produce a single-line JQL or `NONE`.
4. **JQL path:** if valid, `jira_client.search_jql()` runs it (REST v3) → issues become the CONTEXT block.
5. **Fallback:** otherwise the MCP `ai_hub_search` tool runs (retried up to 6×) → documents become the CONTEXT block.
6. `AIClient` → AI gateway: final `_complete()` with the RAG system prompt + CONTEXT + question.
7. Answer flows back → controller → browser as `{response, model, response_time}`.

</details>

### 8.2 Run Case

Running a profile spawns a detached OS process; the UI then polls for running processes and QA Hub suites independently. The runner submits a suite to QA Hub, which triggers a Jenkins build, and writes cache + summary-report files locally.

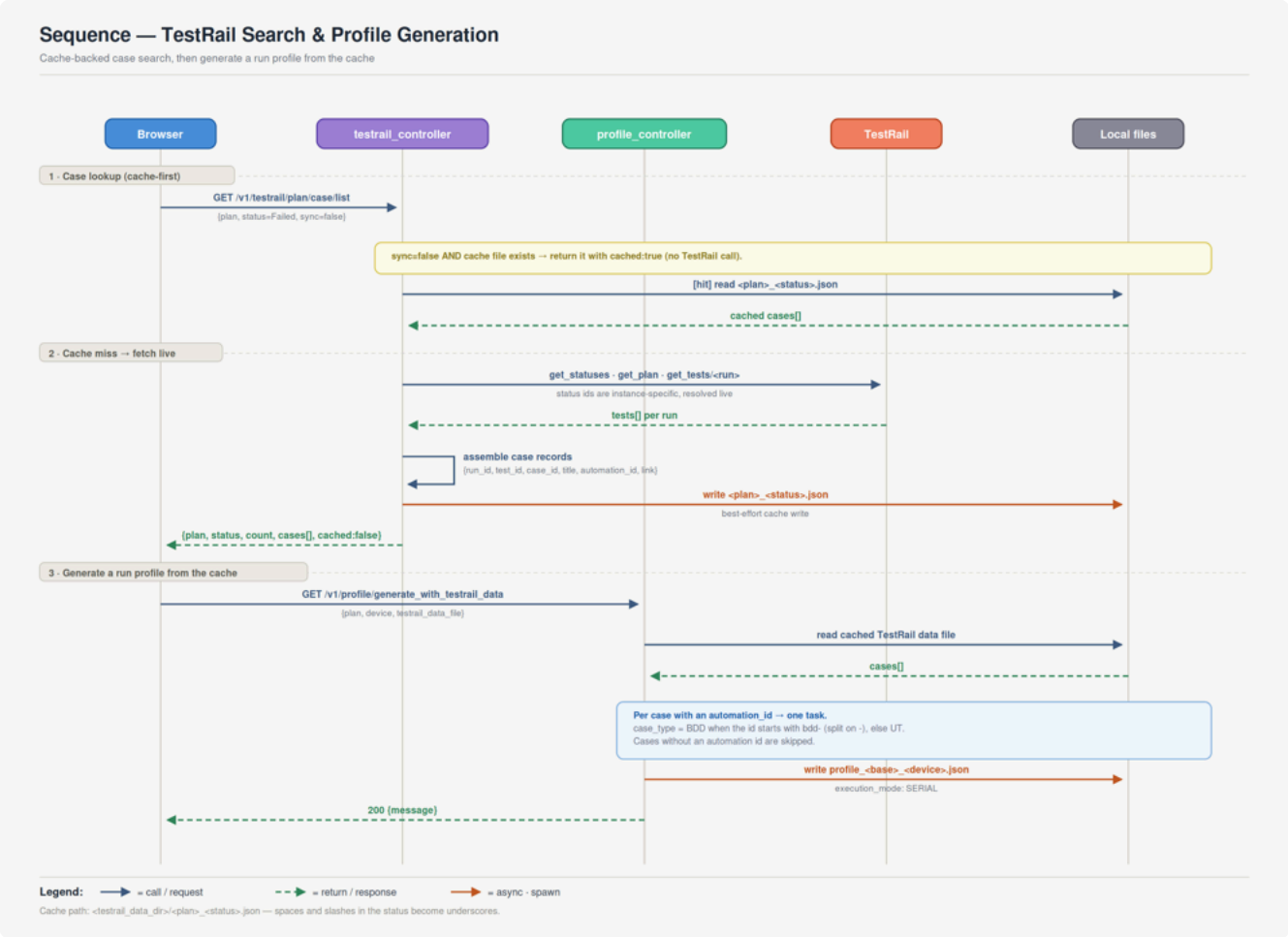



Step-by-step

1. Browser → `runcase_controller`: `POST /v1/runcase/run`. The controller spawns `test_runner.py` in a new session and immediately returns `{message, pid}` — the run is detached.
2. `test_runner.py` → QA Hub: submits a suite; QA Hub triggers a Jenkins build.
3. Browser polls `GET /v1/runcase/list_running` (ps-parsed local processes) and `GET /v1/runcase/list_suites` (QA Hub suites with progress + Jenkins console URL).
4. The runner continuously writes `*.cache` and `*summary_report.txt` files, surfaced under Run Results.

### 8.3 TestRail Search & Profile Generation

TestRail case lookups are cached as `<plan>_<status>.json`. A subsequent "Add Profile with RunPlan" reads that cache file and builds a runnable profile — connecting the TestRail Case tab to the Profile tab.

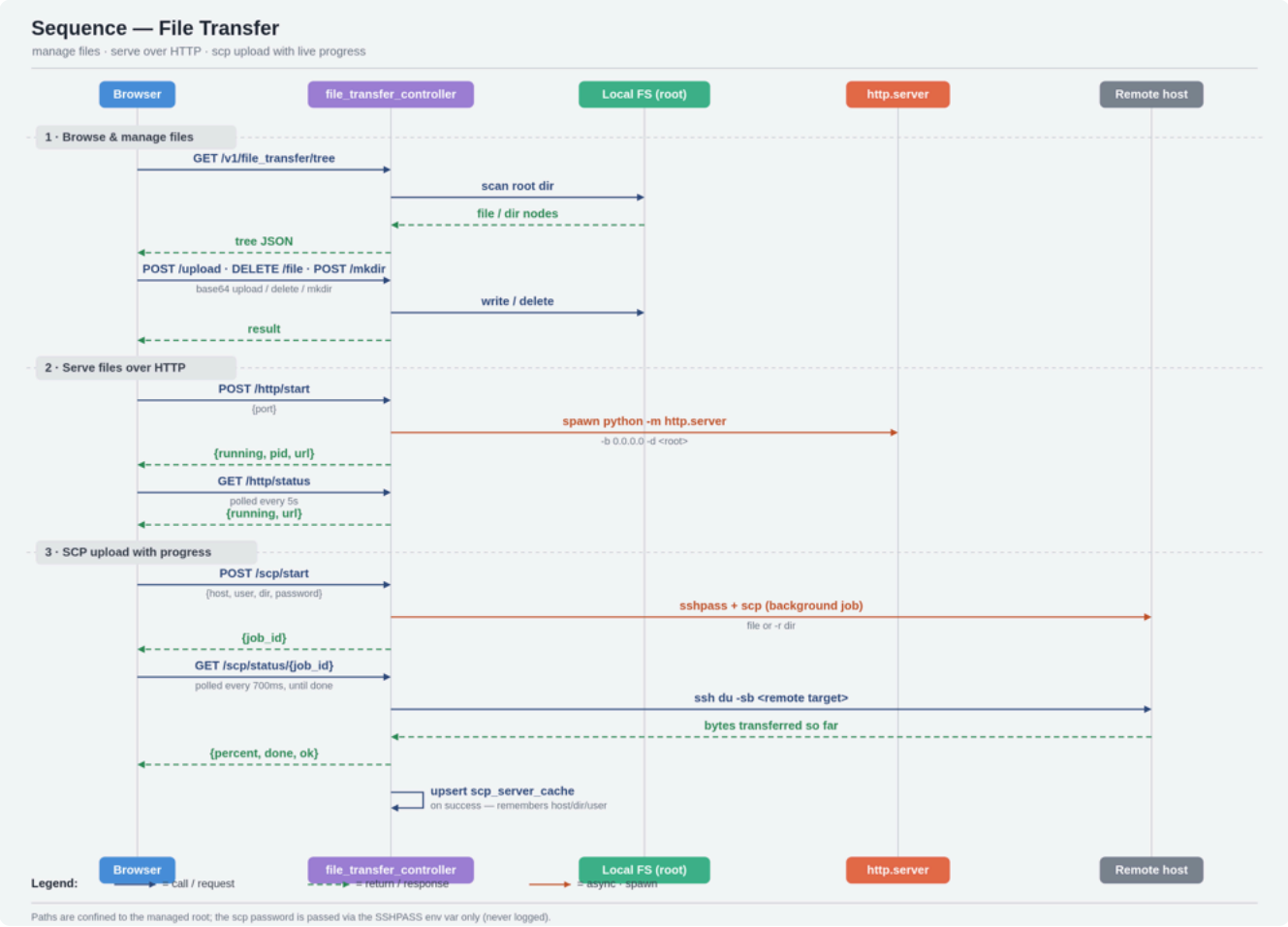



<details>
<summary>Step-by-step</summary>

1. Browser → `testrail_controller`: `GET /v1/testrail/plan/case/list`. With `sync=false` and a cache hit, the cached JSON is returned (`cached:true`).
2. **Cache miss / sync:** the controller resolves the status id, enumerates the plan's runs, pulls the tests, and writes `<plan>_<status>.json`.
3. Later, Browser → `profile_controller`: `GET /v1/profile/generate_with_testrail_data` reads that cache file, builds a task per automated case, and writes `profile_<base>_<device>.json`.
</details>

### 8.4 File Transfer

The File Transfer tab combines local file management, a built-in HTTP file server, and background scp upload with real progress. Browsing/managing files hits the local filesystem under the managed root; Start/Stop spawns a `python -m http.server`; an scp upload runs as a background job whose percent is derived by polling the remote size against the local size, and a successful target is remembered in `scp_server_cache`.



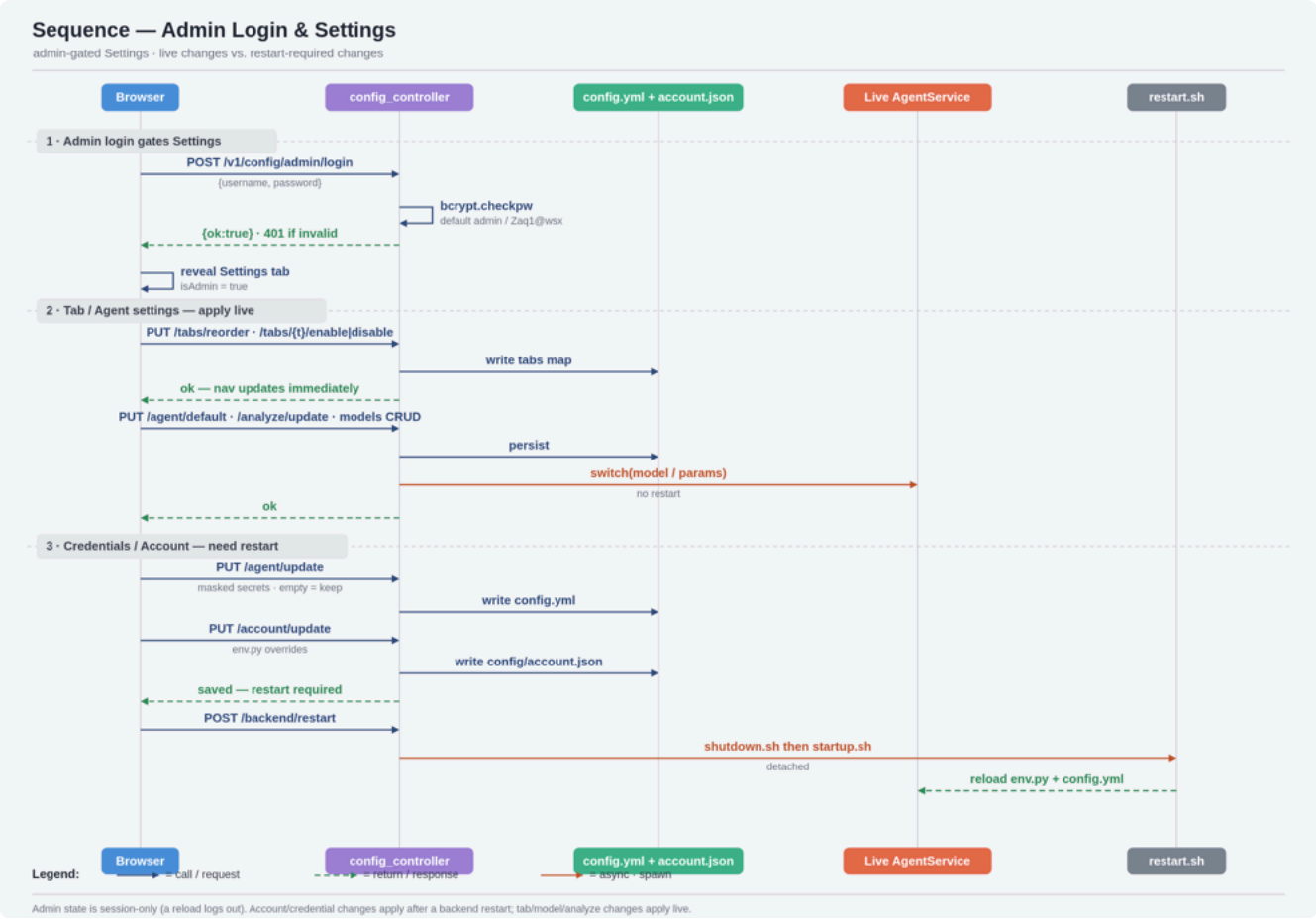

Step-by-step

1. **Browse/manage:** `GET /tree` scans the root; `POST /upload` (base64), `DELETE /file`, `POST /mkdir`, `DELETE /dir` mutate it (all path-traversal guarded).
2. **HTTP server:** `POST /http/start` spawns a singleton `python -m http.server <port> -b 0.0.0.0 -d <root>` (PID tracked in `data/.file_transfer_http.pid`); the UI polls `GET /http/status` and shows a live URL; `POST /http/stop` kills it.
3. **SCP upload:** `POST /scp/start` launches `sshpass + scp` (file or `-r` dir) as a background job and returns a `job_id`; the password is passed via the `SSHPASS` env var only.

4. **Progress:** the UI polls `GET /scp/status/{job_id}`; the backend runs `ssh du -sb` on the remote target and reports `percent` (capped at 99 until the process exits).
5. **Cache:** on success the target (`server/upload_dir/username`) is upserted into `file_transfer.scp_server_cache`, powering the Host autocomplete.

#### 8.5 Admin Login & Settings

The **Settings** tab is gated by **Admin Login**. Once unlocked, tab/agent/analyze changes apply live, while credential and account changes are persisted and take effect after a backend restart.



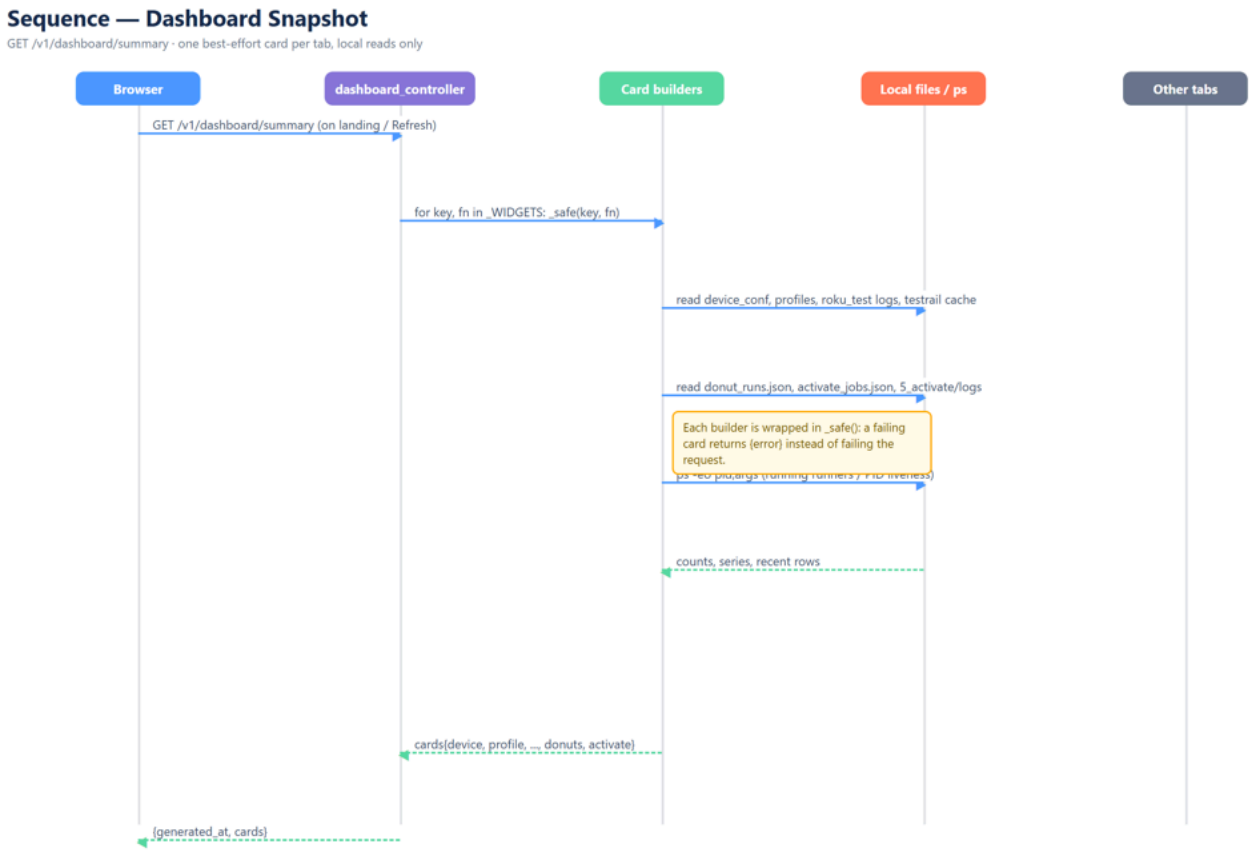

Step-by-step

1. **Login:** `POST /v1/config/admin/login` verifies the password with bcrypt against env.py (default `admin` / `Zaq1@wsx`); on success the UI reveals the Settings tab (`isAdmin`, session-only).
2. **Tab Settings:** `PUT /tabs/reorder` and `/tabs/{t}/enable|disable` write the `tabs` map in config.yml and the nav updates immediately.
3. **Agent Settings:** `PUT /agent/default`, `/analyze/update`, and model CRUD persist to config.yml and live-switch the running AgentService — no restart.

4. **Credentials:** `PUT /agent/update` writes gateway secrets (empty value = keep the stored secret).
5. **Account Settings:** `PUT /account/update` writes the env.py overrides to `config/account.json` (secrets masked; blank = keep).
6. **Apply:** `POST /backend/restart` runs `shutdown.sh` then `startup.sh` detached, reloading env.py + config.yml.

#### 8.6 Dashboard Snapshot

The Dashboard is assembled server-side from local files and process state only, one best-effort card per tab, so it stays fast and never blocks on QA Hub or TestRail.




Step-by-step

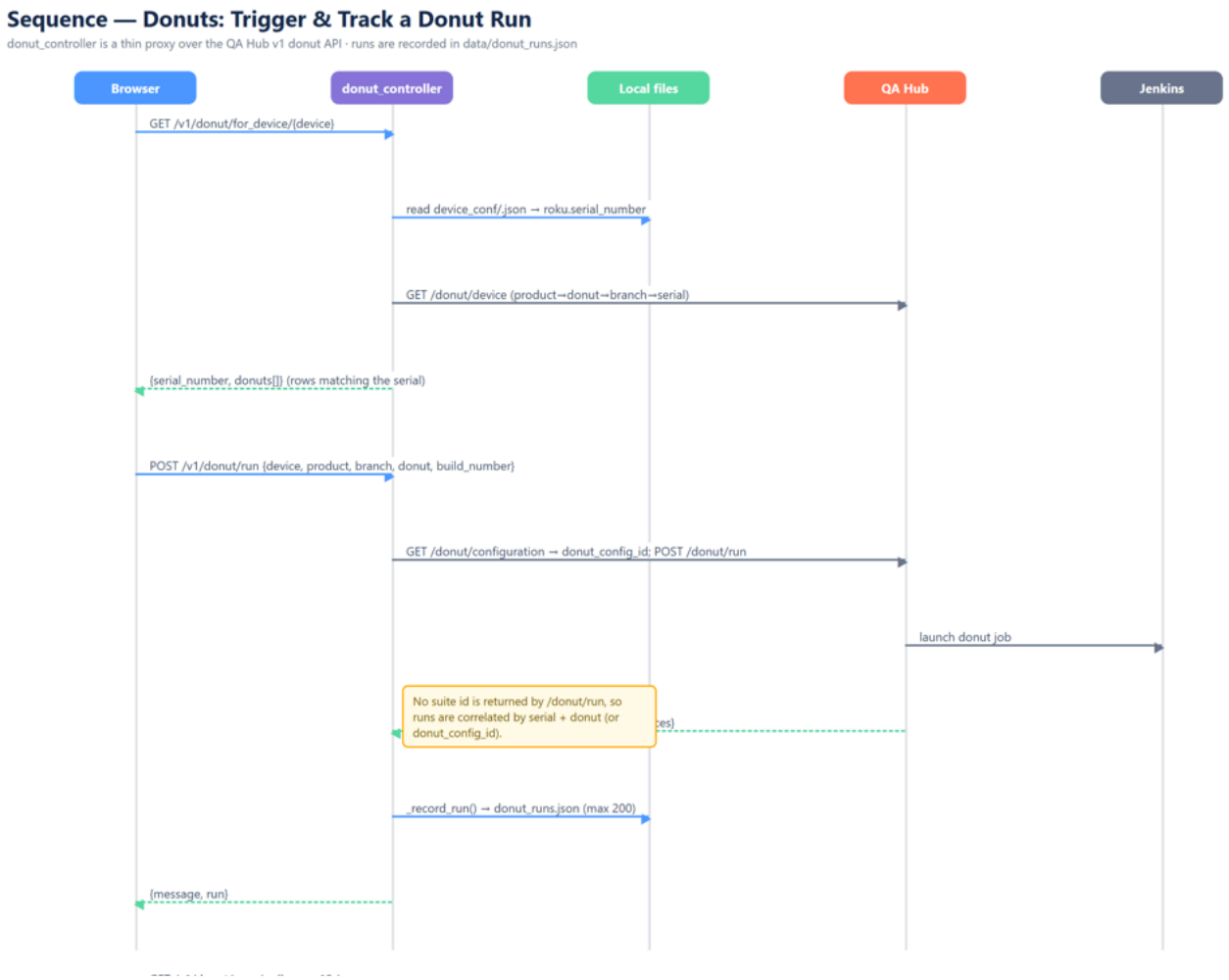
1. Browser → `dashboard_controller` : `GET /v1/dashboard/summary` when Dashboard opens (default landing page) or on **Refresh**.
2. The controller runs every builder in `_WIDGETS` ( `device, profile, runcase, rokutest, analyzehistory, testrail, agent, reference, filetransfer, donuts, activate` ), each wrapped in `_safe()` .
3. Builders read only local data: `config/device_conf` , `4_runcase/profiles` , roku_test logs, the TestRail cache, analyze history, `data/donut_runs.json` ,

`backend/conf/activate_jobs.json` and `5_activate/logs`; running processes come from `ps`. No external HTTP.

4. A failing builder returns `{error}` for its own card only; the response is `{generated_at, cards}`.
5. The SPA renders only the cards whose tab is in `visibleTabs`; `GET /v1/dashboard/card/{name}` refreshes a single card.
6. Clicking a card navigates to its tab.

#### 8.7 Donuts: Trigger & Track

A donut run is launched through QA Hub, which hands it to Jenkins asynchronously. Zen records each launch locally and correlates it with the live Jenkins / queued-run lists to show status.




Step-by-step

1. `GET /v1/donut/for_device/{device}`: the serial is resolved from `config/device_conf/<device>.json` (`roku.serial_number`); QA Hub `GET`

`/donut/device` returns a product→donut→branch→serial tree, flattened to the donuts this device is registered on.

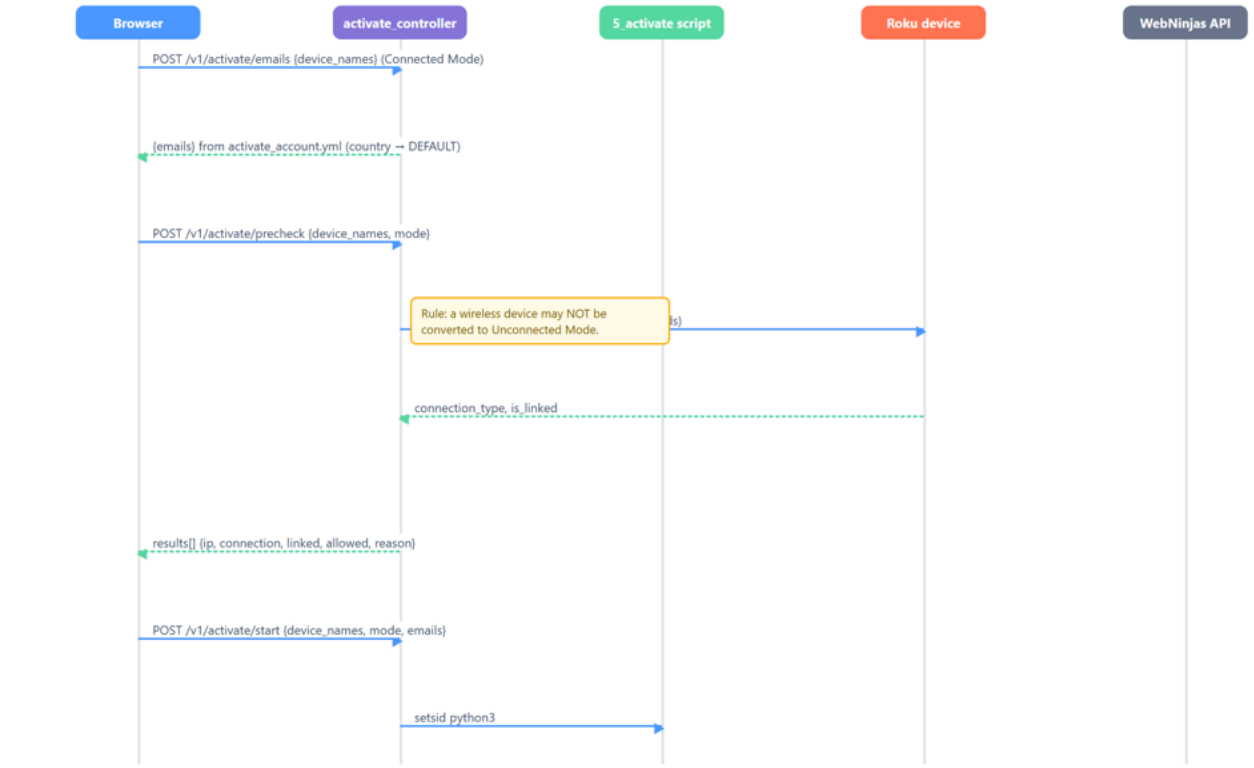
2. `POST /v1/donut/run` {device, product, branch, donut, build_number}: the controller looks up `donut_config_id` via `GET /donut/configuration/{product}/{branch}/{donut}`, then POSTs `/donut/run` with a single-element `serial_numbers`.
3. QA Hub answers **200** (triggered; QA Hub launches the Jenkins donut job) or **202** with `queued_run_id` / `busy_devices` when the device is busy.
4. The launch is recorded in `data/donut_runs.json` (newest first, max 200).
5. The UI polls `GET /v1/donut/runs` every 10s. Because `/donut/run` returns no suite id, each run is correlated by serial + donut (or `donut_config_id`) against `donut/jenkins-running` and `donut/queued-runs` (queried once per product/branch).
6. Actions: `POST /v1/donut/abort/job` (→ `DELETE donut/jenkins-running`), `POST /v1/donut/abort/queued` (→ `DELETE donut/queued-runs`), `GET /v1/donut/results/{build}` for finished runs, `DELETE /v1/donut/runs/{id}` to drop the local record only.

#### 8.8 Activate: Connected / Unconnected Mode

Devices are prechecked, then each allowed device is converted by its own detached `5_activate` process; the UI follows progress through the job list and live logs.

## Sequence — Activate: Convert Devices to Connected / Unconnected Mode

POST /v1/activate/precheck → /v1/activate/start → one detached 5_activate script per device




Step-by-step

1. Connected Mode: `POST /v1/activate/emails` pre-fills each device's activation account from `config/activate_account.yml` (first country key contained in the device name, else `DEFAULT`); the user may override it.
2. `POST /v1/activate/precheck` {device_names, mode}: per device (in parallel threads) the IP is pinged and `5_activate/precheck_connection.py` reports `connection_type` and `is_linked`.
3. Rule: a **wireless** device may not be converted to **Unconnected Mode**; such rows come back `allowed:false` with a reason.
4. `POST /v1/activate/start` {device_names, mode, emails}: devices with a running job or a failed precheck are `rejected`; every other device gets its own `setsid python3 <script> <device> [email]` process and log file (`<mode>_activate_<device>_<ts>.log`). Jobs are stored in `backend/conf/activate_jobs.json`.
5. Script: `factory_reset_and_activate.py` (Connected — factory reset, guided setup, then WebNinjas `POST /api/1.0/activate` {esn, email} once the activation screen appears; 600s timeout) or `factory_reset_and_unconnected_mode.py` (Unconnected — guided setup picks "connect later"). Both finish by disabling auto power-down and enabling fast TV start.

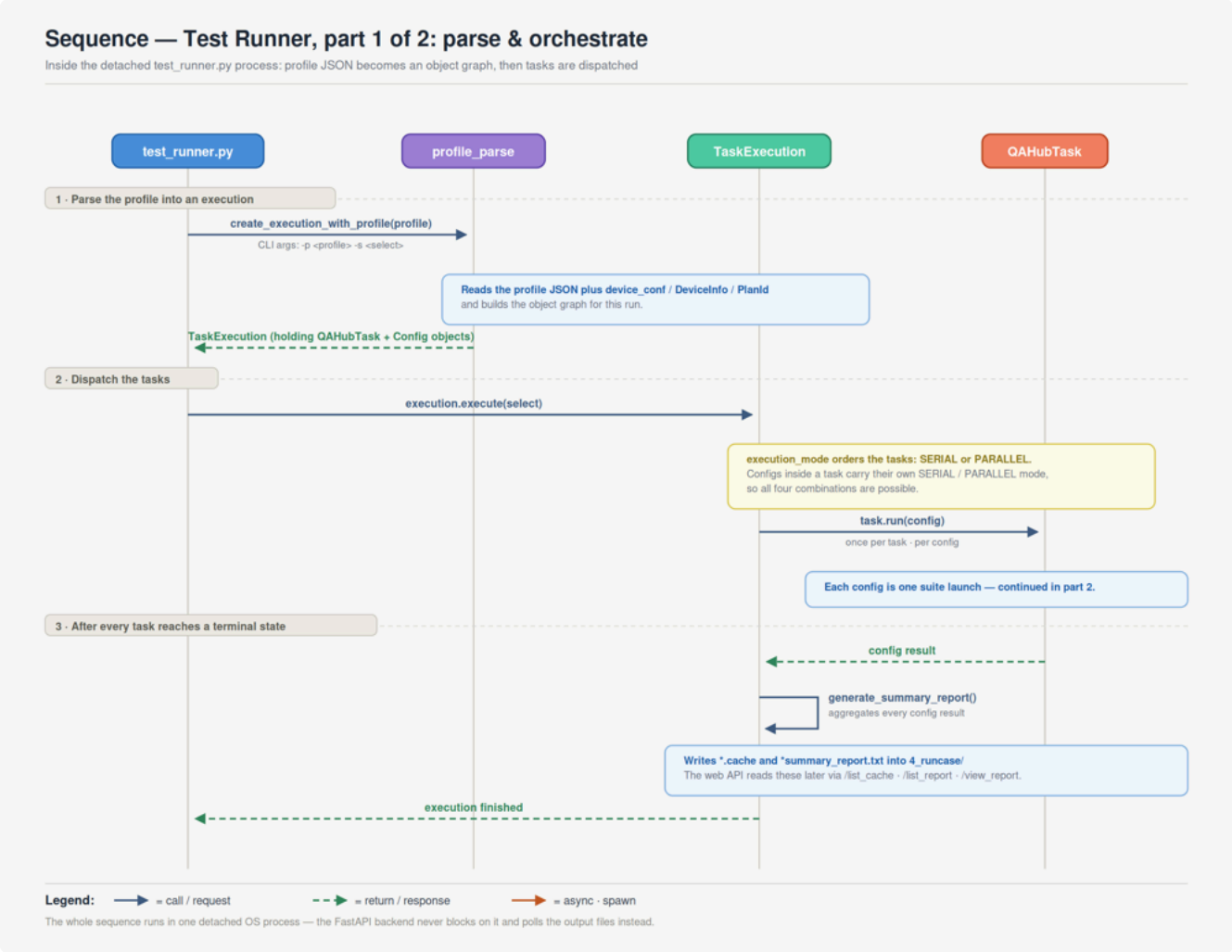
6. The UI polls `GET /v1/activate/jobs` and tails logs with `GET /v1/activate/log/view/{log_file}` (last 5000 lines); `POST /v1/activate/stop/{pid}` stops a job, `DELETE /v1/activate/jobs/{id}` removes a finished one.

### 9. Test Runner & Data Model (4_runcase)

The Run Case and Profile features are built around the `4_runcase` runner and its **profile** data model.

#### 9.1 Component call relationships

The backend spawns `test_runner.py` as a detached process (POST /v1/runcase/run). Inside that process, these modules collaborate to parse the profile, submit a suite to QA Hub, poll it to completion, and write result files:




| Component | Responsibility |
| --- | --- |

| | |
|---|---|
| `test_runner.py · Runner` | CLI entry ( `-p profile -s select` ); calls `create_execution_with_profile()` then `.execute(select)` . |
| `profile_parse.py` | Reads the profile JSON (plus `device_conf` / `DeviceInfo` / `PlanId` ) and builds a `TaskExecution` via `_parse_task()` → `_parse_config()` . Each task has exactly one `config` ; defaults are `stop_if_success=True` , `timeout=36000` , `test_source="file-system"` , branches `main` , `testrail_plan_id=PlanId.NO_PLAN` . |
| `task_handle.py · TaskExecution` | Orders tasks (SERIAL = sequential, PARALLEL = one thread per task), runs `execute(select)` , maintains the `.cache` file ( `task_results` + `task_progress` ) and writes the summary report. |
| `task_handle.py · QAHubTask` | Runs the task's single config: `triger_test_suite` → `track` . A `Queued` suite is terminated and retried up to `QUEUED_RETRY_COUNT=3` times 60 s apart; the suite is polled every `TRACK_RESULT_INTERVAL=10` s until the task's `timeout` runs out. |
| `launch_qahub_testcase.py` | `launch_test` → `build_payload` → `post_suite` |

| | |
|---|---|
| | (POST /suite, Token auth); also `terminate_suite`. |
| `track_pipeline.py` | `track_result` → `get_suite` (poll state); `get_build_url` resolves the Jenkins console URL. |

#### 9.2 Profile structure

```
{
  "name": "execution sample profile",
  "description": "execution sample profile",
  "execution_mode": "SERIAL",            // or PARALLEL - how the tasks run
  "tasks": [
    {
      "config": {                        // exactly ONE config per task
        "case_type": "BDD",              // BDD | UT | NEW
        "case_id": "2659886400",         // BDD: numeric id (no bdd- prefix)
                                         // UT/NEW: "TestClass.test_method"
        "testrail_case_id": 2659886400,  // used for upload + analysis naming
        "device": "d_de_2",              // device config file name (no .json)
        "testrail_plan_id": 0,           // 0 = no push, or a plan id
        "repeat": 0,                     // extra repeats (2 = 3 runs)
        "automation_test_branch": "main",
        "roku_automation_branch": "main"
      },
      "timeout": 36000                   // tracking timeout in seconds
    }
  ]
}
```

⚠ **Format change.** A task now carries a single `config` object instead of a `configs` list, and the per-task `execution_mode`, `stop_if_success` and `print_console_message` keys are no longer managed — `profile_controller._strip_dropped_task_keys()` removes them (plus config `test_source`) from everything written through the API/UI, and `profile_parse.py` supplies its own defaults (`stop_if_success=True`, `timeout=36000`). Hand-written profiles that still set those keys keep working.

#### 9.3 Execution modes

Only the profile-level `execution_mode` is honoured now that a task holds one config:

| execution_mode | Behaviour |
|---|---|
| `SERIAL` | Tasks run one at a time; each is launched and tracked to completion before the next starts. |
| `PARALLEL` | Every task is launched on its own thread and tracked concurrently; `execute()` joins them at the end. |

**Selection & cache.** `execute(select)` takes the same values as the CLI `-s` flag ( `All` , `Fail` , `Pass` , `Skipped` , `Blocked` , `NA` , `Unknown` , `NoTest` ). `All` clears the cache and runs everything; `NoTest` skips cases already present in the cache; any other value runs only the cached tasks whose status matches.

**Task statuses** ( `util.TaskStatus` ): Pending, Processing, Setup, Queued, Running, Pass, Failed, Blocked, Skipped, NA, Unknown, Aborted.

**Outputs.** `data/runcase_report/<ts>_<profile>.cache` ( `task_results` keyed by case ids + `task_progress` "n/m", rewritten after every task) and `<ts>_<profile>_summary_report.txt` ( `{profile_name, count, tasks: [{case_ids, testrail_case_ids, status, log_url, build_number}]}` ) — the file auto analyze (§5.3) consumes. Suite names are `[Zen]_<device>_<case>_<plan>_<repeat>` and launches are appended to `4_runcase/logs/runcase_history.txt` .

#### 9.4 Device config model

Device files live in `config/device_conf/` ; the file name (minus `.json` ) is the device key used by profiles. Fields under a `roku` object:

| Field | Required | Description |
|---|---|---|
| `ip_address` | Yes | Device IP |
| `brand_name` | Yes | TV brand |
| `device_id` / `serial_number` | Yes | Device identifiers |
| `maxwell_config.host` | No | Tuner simulator IP |

| | | |
|---|---|---|
| `power_config` | No | Power strip: `host`, `outlet`, `model` |
| `qa_bot_config.host` | No | QA Bot (Raspberry Pi) IP |
| `rtvsp_config` / `serial_config` | No | IR blaster / serial console |

### 10. TestRail Caching

TestRail lookups are cached to keep the UI responsive and to feed the profile generator.

- **Cache file:** `<testrail_data_dir>/<plan>_<status>.json` (status spaces/slashes → `_`).
- **Read path** (`plan/case/list`): if `sync=false` and the cache file exists, return it with `cached:true`. Otherwise fetch fresh: resolve the `status_id` live, enumerate the plan's run ids (`get_plan` → entries → runs), pull `get_tests/<run>&status_id=<id>` for each, assemble case records with per-test view links, write the cache (best-effort), and return with `cached:false`.
- **Consumers:** `testrail_data/available` and `/list` expose which caches exist; `profile/generate_with_testrail_data` reads a cache file to build a run profile.

### 11. External Integrations

| Service | Endpoint / Base | Auth | Used for |
|---|---|---|---|
| AI gateway (ai-hub-lite) | `https://ai-hub-lite.msc.rokulabs.net/v1` + `/mcp-rest` | Bearer api_key + X-Api-Key-Id/Alias | chat completions; MCP `ai_hub_search` |
| TestRail | `testrail.eswat.roku.com/testrail` API v2 | Basic (email + API key) | plans, runs, tests, statuses |
| JIRA | `roku.atlassian.net` REST v3 | Basic (email + token) | JQL queries for the assistant |

| | | | |
|---|---|---|---|
| QA Hub | `qahub.eswat.rokulabs.net/api/v1` | Token header | suite list / terminate / detail; donut catalogue, run, jenkins-running, queued-runs, results; device reservation |
| Jenkins | `jenkins.eswat.rokulabs.net/job/qa_hub_launch_suite` | — | build console URL from suite build number |
| Git (local) | `automated_tests` repo | local | branch list / current / checkout |
| WebNinjas | `webninjas.portal.roku.com/api/1.0/activate` | — | account activation (by ESN + email) in Connected Mode, from `5_activate` |

⚠ GitLab appears only as a selectable *source label* for the assistant's semantic search; the backend performs no direct GitLab API calls. Repository operations use the local git CLI.

### 12. Cross-Cutting Concerns

| Concern | Design |
|---|---|
| CORS | Fully open on the backend (all origins/methods/headers) to simplify the SPA-to-API dev proxy. |
| Logging | Shared `LogUtil` logger, console-only (file handler removed; stray |

| | |
|---|---|
| | empty `backend.log` deleted at startup). |
| Configuration | Backend via `conf/config.py` (env → YAML precedence). Frontend host/port + tab visibility via `frontend/conf/config.yml` injected at build time. |
| Error handling | Controllers raise `HTTPException` with descriptive detail; the SPA surfaces `detail` as a toast. |
| Process model | Runs are detached OS processes (Run Case, Roku Test, Activate); the UI tracks them by PID and by parsing `ps`. |
| Secrets | Credentials come from `config/env.py` / environment; never returned by the API (only `key_alias` is surfaced). |

---

*Zen Function Design — generated from source at* `QATools`*. Excludes* `1_create_test_plan` *per scope;* `5_activate` *is covered via the Activate tab.*